

第5章 逻辑数据库设计



第5章 逻辑数据库设计



E-R 转模式



函数依赖

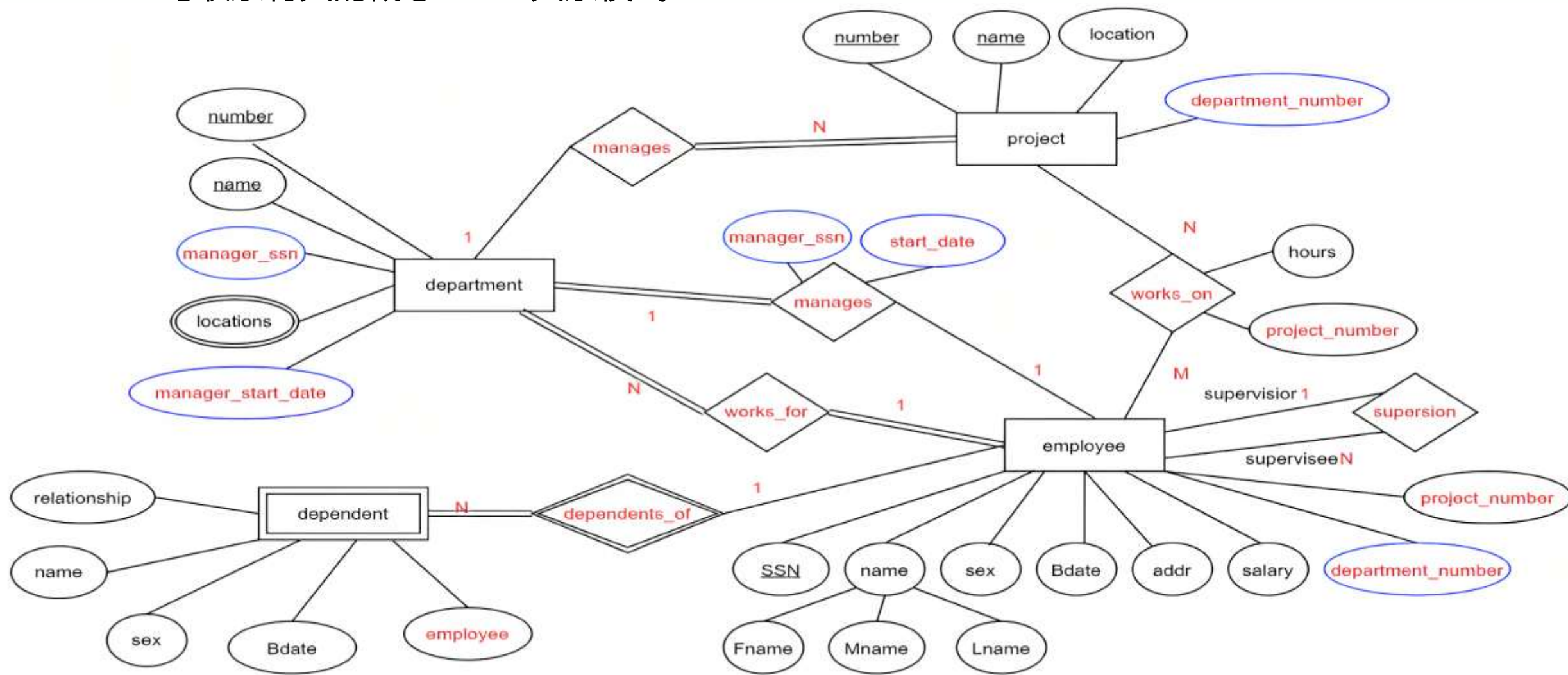


关系范式



模式分解

- 与实体有关的概念 → 关系模式
- 与联系有关的概念 → 关系模式

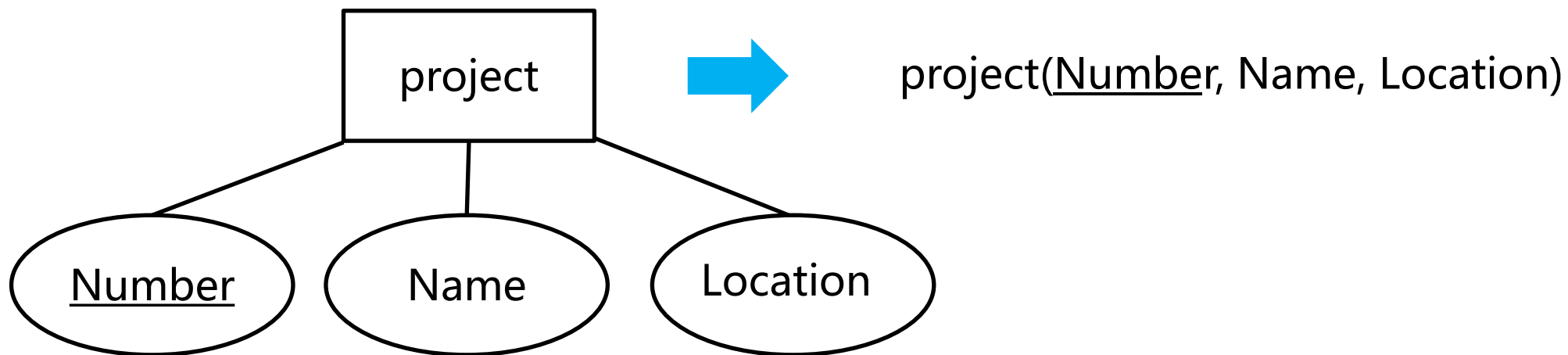


实体型的转换

■ 实体型的转换

转换规则

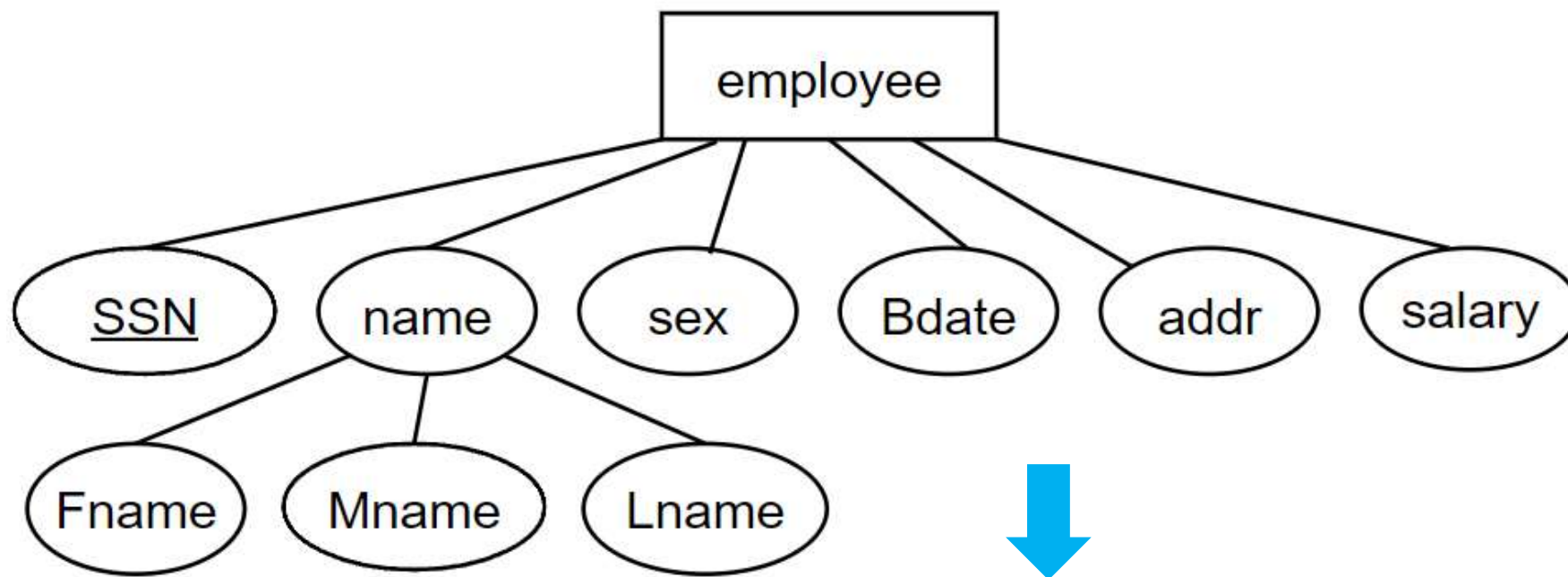
- 实体型的名称 → 关系名
- 实体型的属性集 → 关系的属性集（多值属性除外）
- 实体型的主键 → 关系的主键
- 实体 → 元组



■ 复合属性的转换

转换规则

- 复合属性的最低级组成成分 → 关系的属性

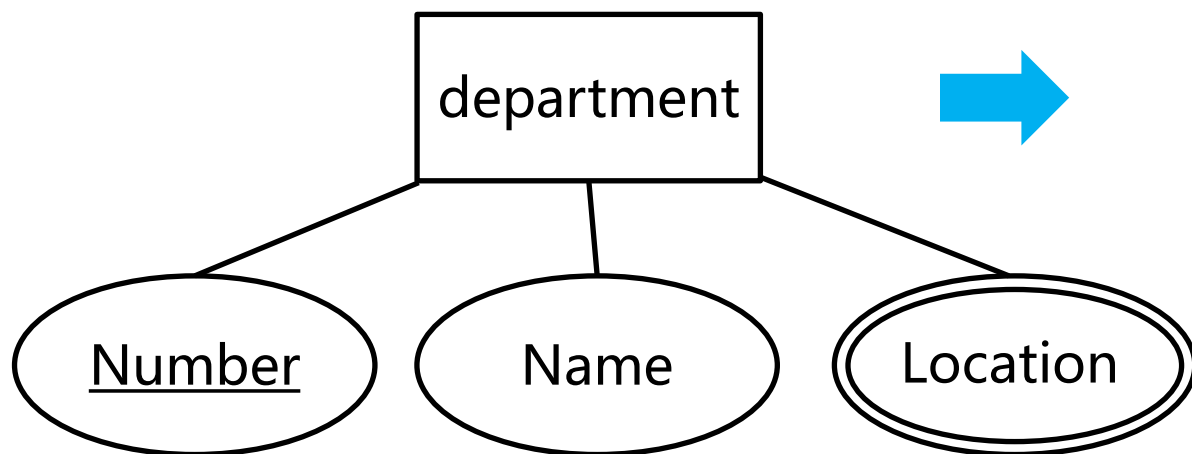


employee(SSN, Fname, Mname, Lname, sex, Bdate, addr, salary)

■ 多值属性的转换

转换规则

- 实体型关系的属性集中不包含多值属性
- 多值属性 → 关系
- 将实体型的主键并入多值属性关系中
- 建立多值属性关系到实体型关系的外键约束

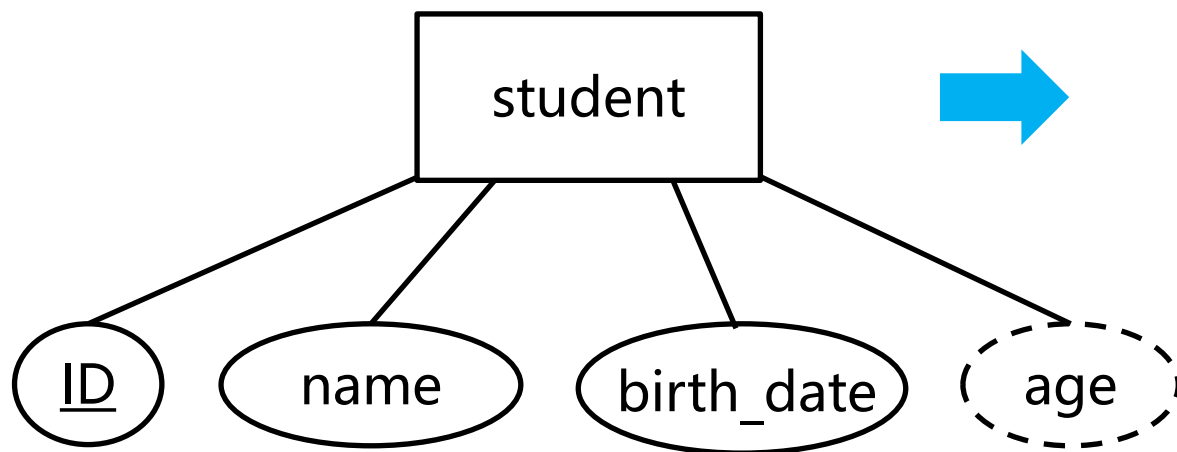


department(Number, Name)
location(Number, Location)

■ 派生属性的转换

转换规则

- 实体型关系的属性集中不包含派生属性
- 派生属性 → 自定义函数



student(ID, name, birth_date)
age(birth_date)

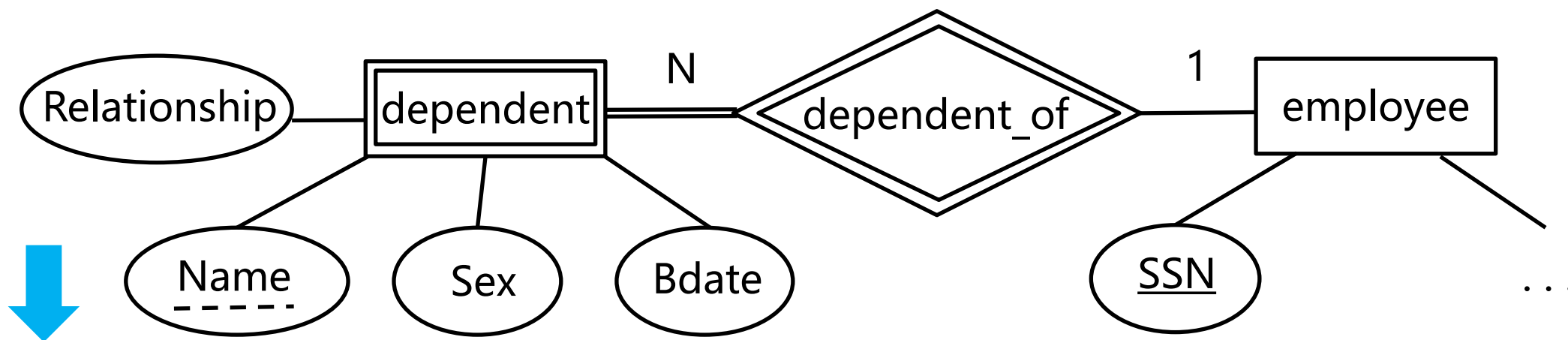
```
delimiter $  
create function age(birth_date datetime)  
returns float(3,1)  
begin  
    return (datediff(now(), birth_date) / 365);  
end $  
delimiter ;
```

```
select ID, name, age(birth_date) as age  
from student;
```

■ 弱实体型的转换

转换规则

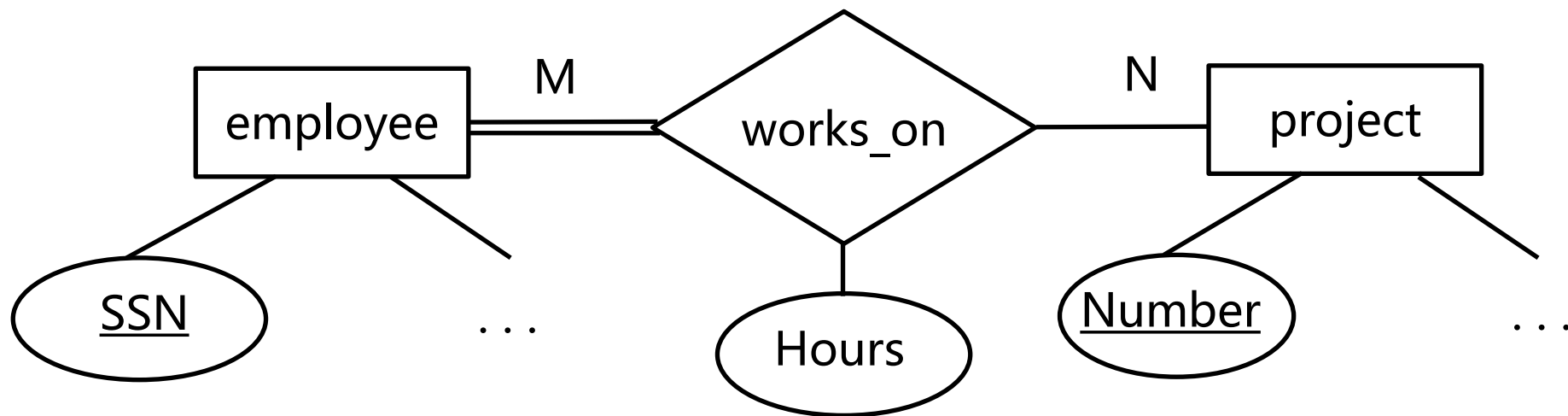
- 弱实体型的名称 → 关系名
- 弱实体型的属性集 U 属主实体型的主键 → 关系的属性集
- 弱实体型的部分键 U 属主实体型的主键 → 关系的主键
- 建立弱实体型关系到属主实体型关系的外键约束



dependent(SSN, Name, Sex, Bdate, Relationship)

联系型的转换

■ M:N 二元联系型的转换



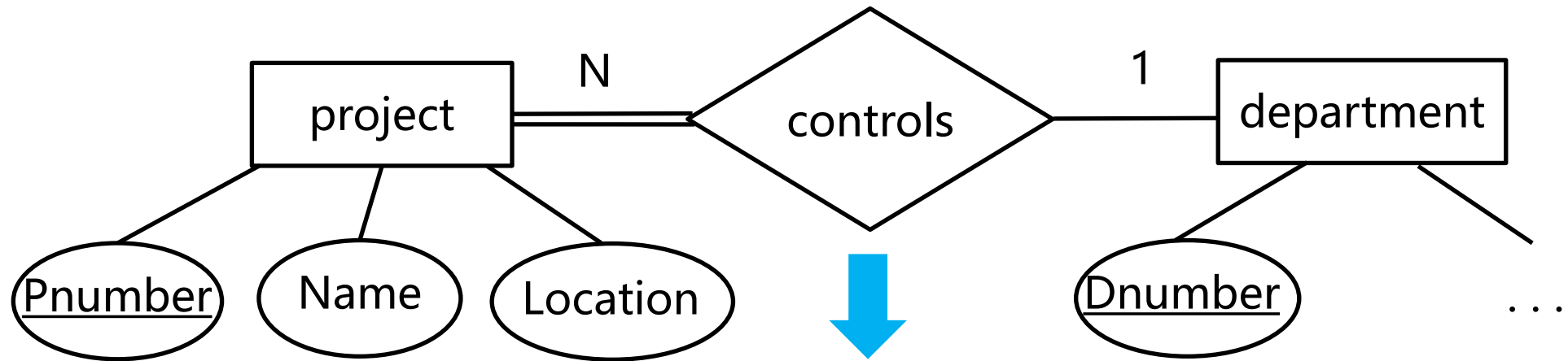
works_on(SSN, Number, Hours)

■ M:N 二元联系型的转换

转换规则

- 联系型的名称 → 关系名
- 实体型 1 的主键 U 实体型 2 的主键 U 联系型的属性集 → 关系的属性集
- 实体型 1 的主键 U 实体型 2 的主键 → 关系的主键
- 建立联系型关系到实体型 1 关系的外键约束
- 建立联系型关系到实体型 2 关系的外键约束

■ N:1 二元联系型的转换



project(Pnumber, Name, Location)

controls(Pnumber, Dnumber)



project(Pnumber, Name, Location, Dnumber)

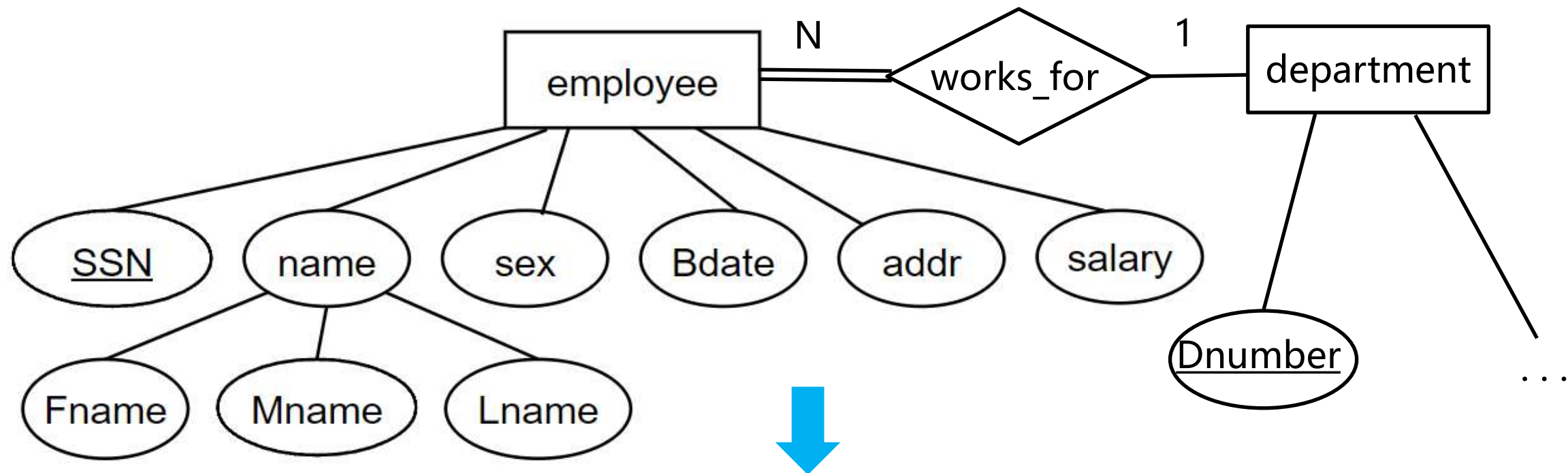
foreign key (DNumber) references department(Number)

■ N:1 二元联系型的转换

转换规则

- 设联系型中基数 N 一方的实体型（如 employee）为实体型 1
- 基数 1 一方的实体型（如 department）为实体型 2
- 将实体型 2 的主键和联系型的属性并入实体型 1 的属性集中
- 建立实体型 1 关系到实体型 2 关系的外键约束

■ N:1 二元联系型的转换



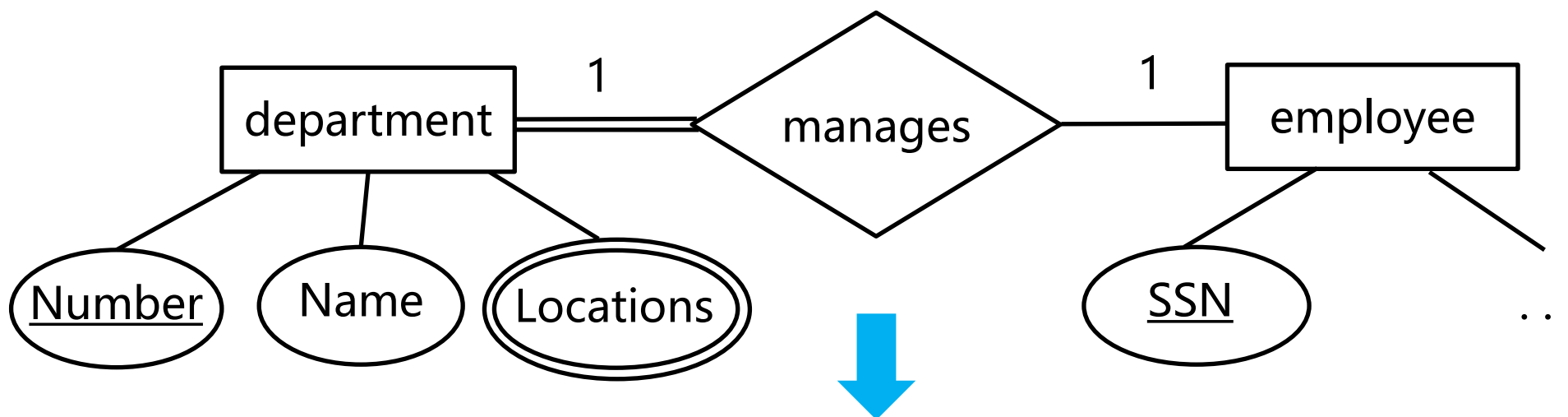
employee(SSN, FName, Mname, Lname, sex, Bdate, addr, salary, **Dnumber**)

DNumber 参考 department.Number

■ 1:1 二元联系型的转换

转换规则

➤ 与 N:1 联系的转换方法相同



方案1: employee(SSN, ..., Dnumber, Start_date)

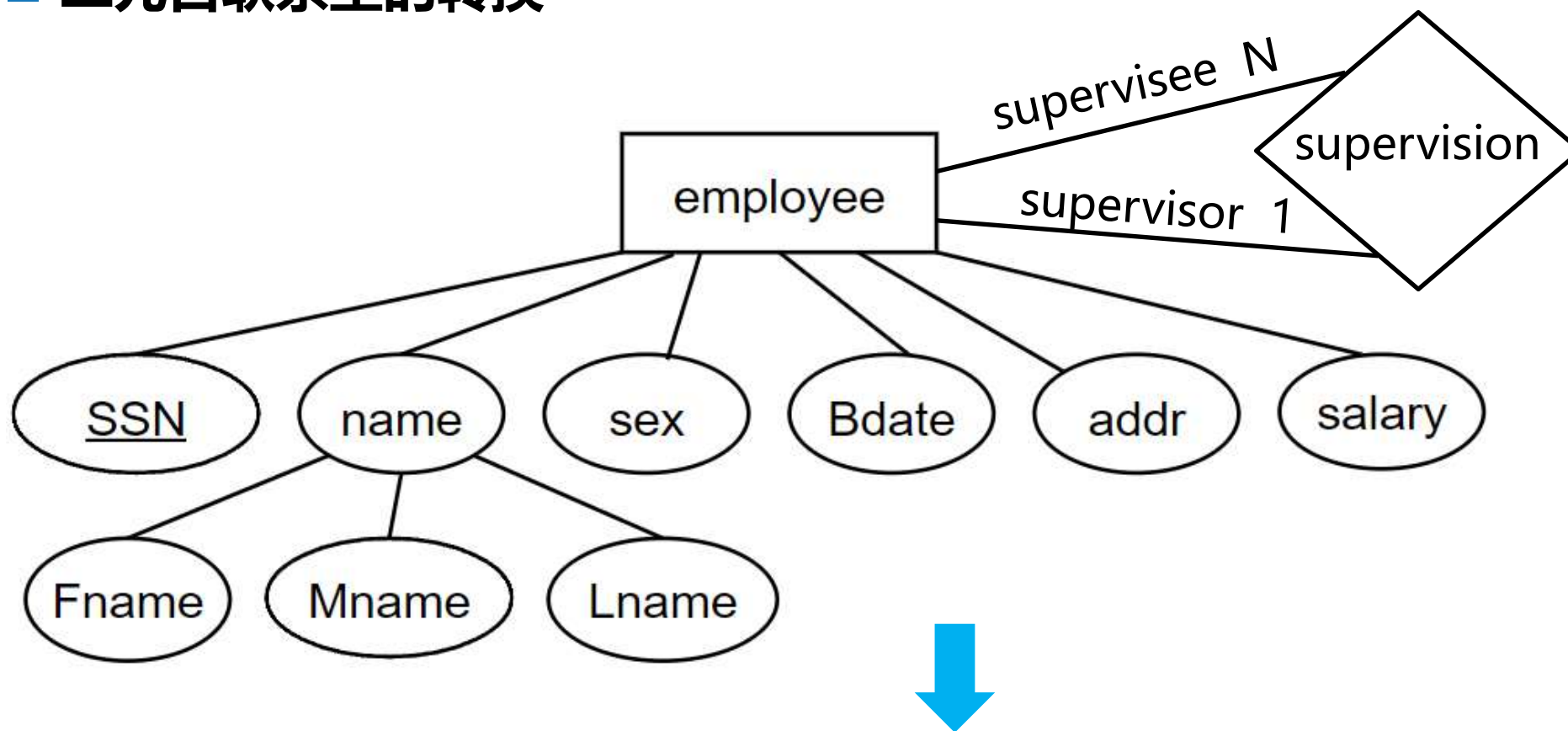
vs 方案2: department(Number, Name, SSN, Start_date)

■ 二元自联系型的转换

转换规则

- 与不同实体型间二元联系型的转换方法相同
- 对重名属性重命名

■ 二元自联系型的转换

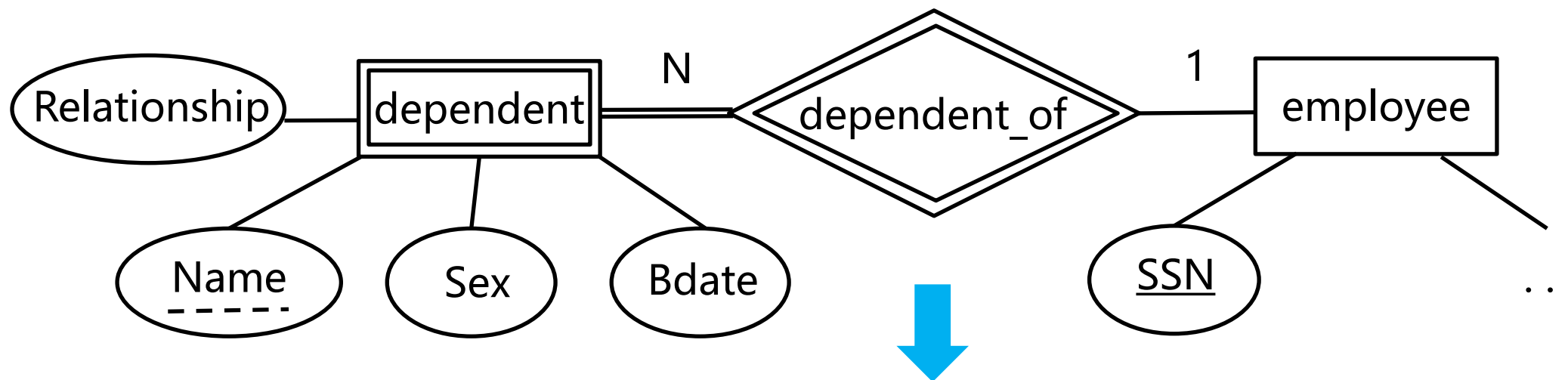


employee(SSN, FName, Mname, Lname, sex, Bdate, addr, salary, SupervisorSSN)

■ 标识联系型的转换

转换规则

- 由于弱实体型转换成的关系已经包含了属主实体型的主键
- 因此标识联系型就不必转换了



dependent(SSN, Name, Sex, Bdate, Relationship)

■ 公司案例

➤ 实体型和 N:1 联系型转换成的关系模式

- EMPLOYEE(SSN, Fname, Mname, Lname, Sex, Bdate, Address, Salary, Number, SupervisorSSN)
- PROJECT(Number, Name, Location, DNumber)
- DEPARTMENT(Number, Name, SSN, Start_date)
- DEPENDENT(SSN, Name, Sex, Birth_date, Relationship)

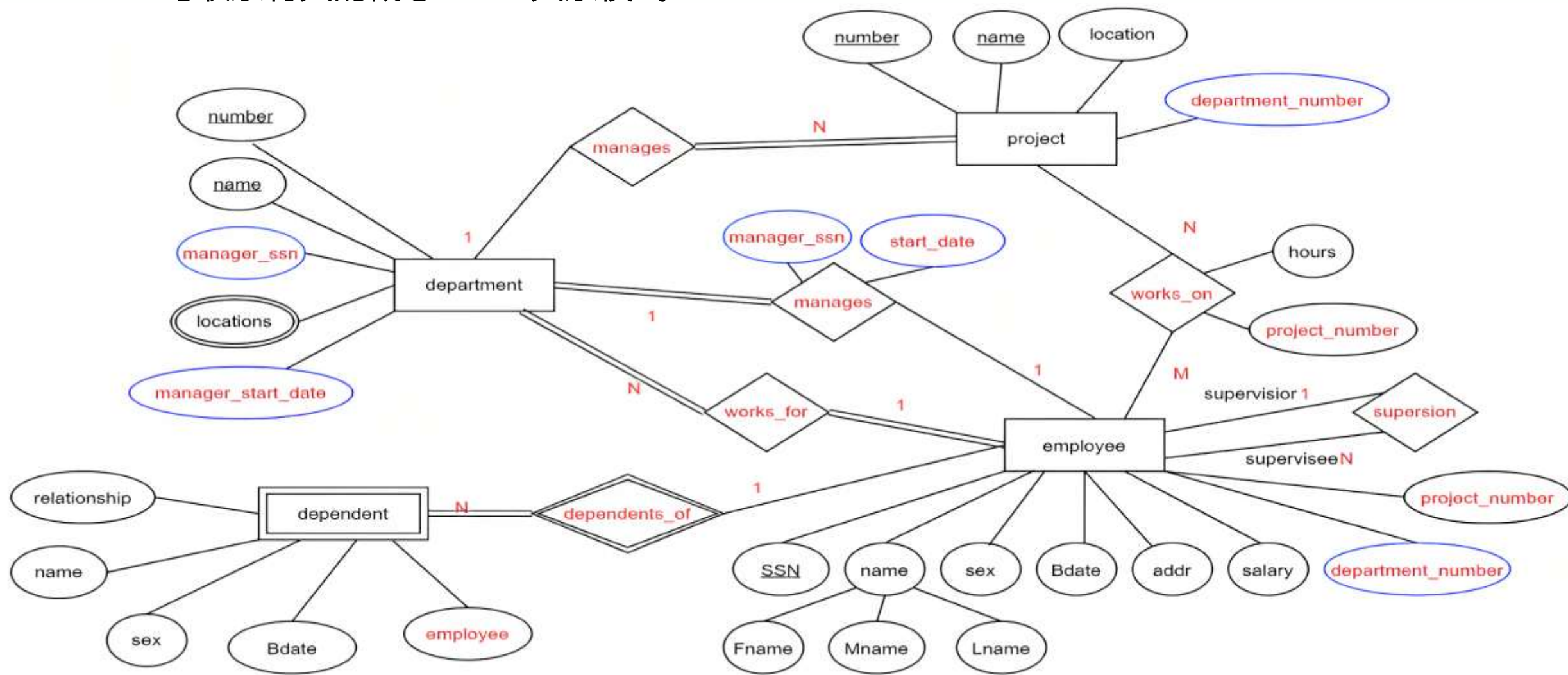
➤ 多值属性转换成的关系模式

- Locations(Number, Location)

➤ M:N 联系型转换成的关系模式

- WORKS_ON(SSN, Number, Hours)

- 与实体有关的概念 → 关系模式
- 与联系有关的概念 → 关系模式



■ 关系数据库模式的评估

- 目的：评价一个关系数据库模式设计的“好坏”
- 方法：用关系数据库规范化理论来评估
 - 关系模式越规范，数据更新时产生的问题越少，因此就越“好”
 - 关系模式越不规范，数据更新时产生的问题越多，因此就越“差”
- 注意：
 - 关系数据库规范化程度不是评估关系数据库模式的唯一准则
 - 某些情况下，规范的关系模式在数据库系统中的实际表现并不好
 - 要“因地制宜”

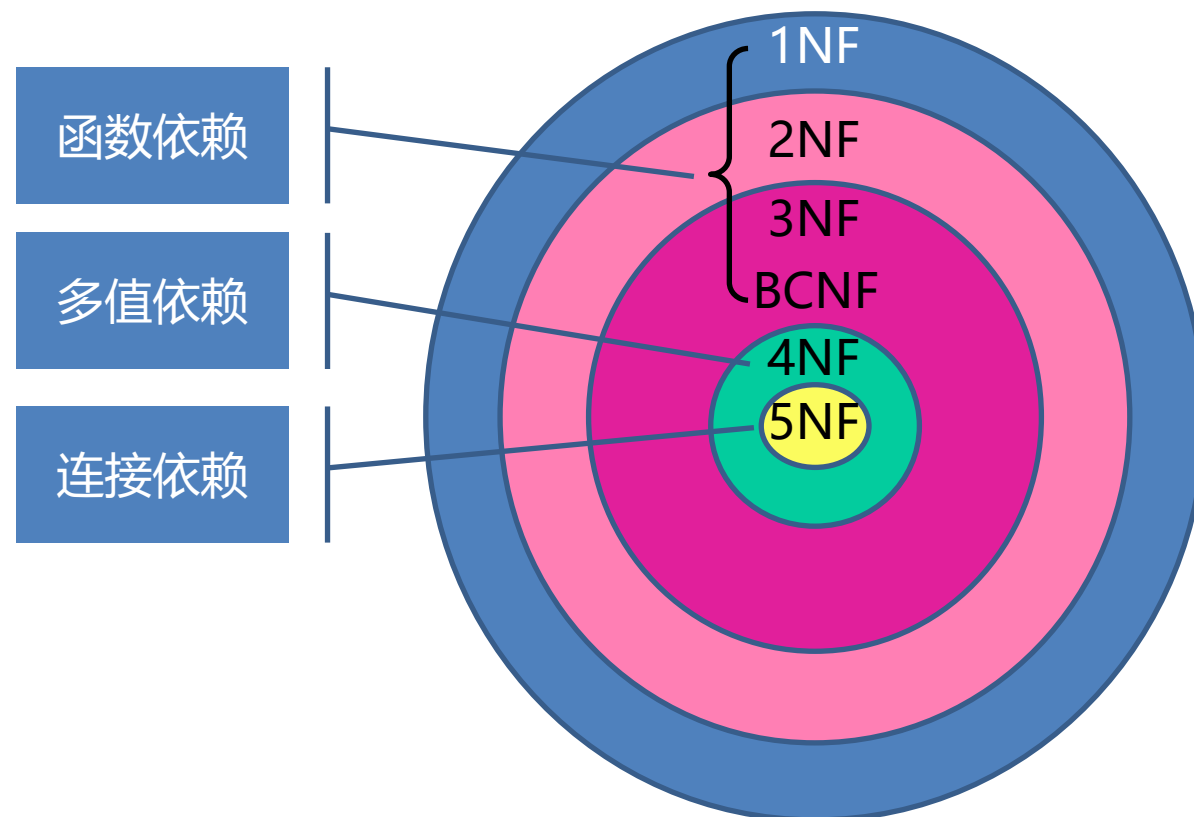
■ 数据库的规范性设计

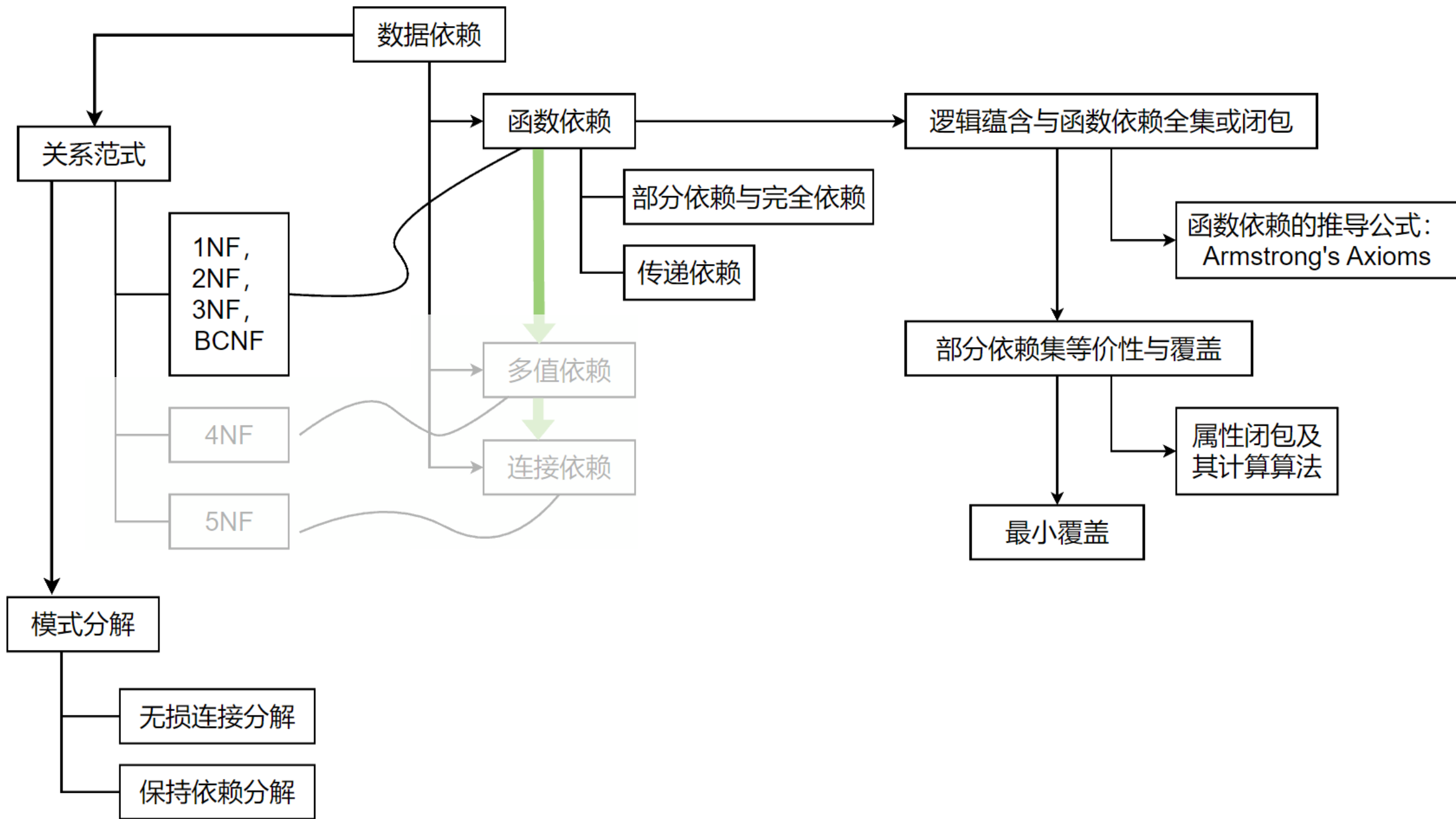
数据库的规范性设计需要分析关系中的属性在取值方面有什么依存关系？

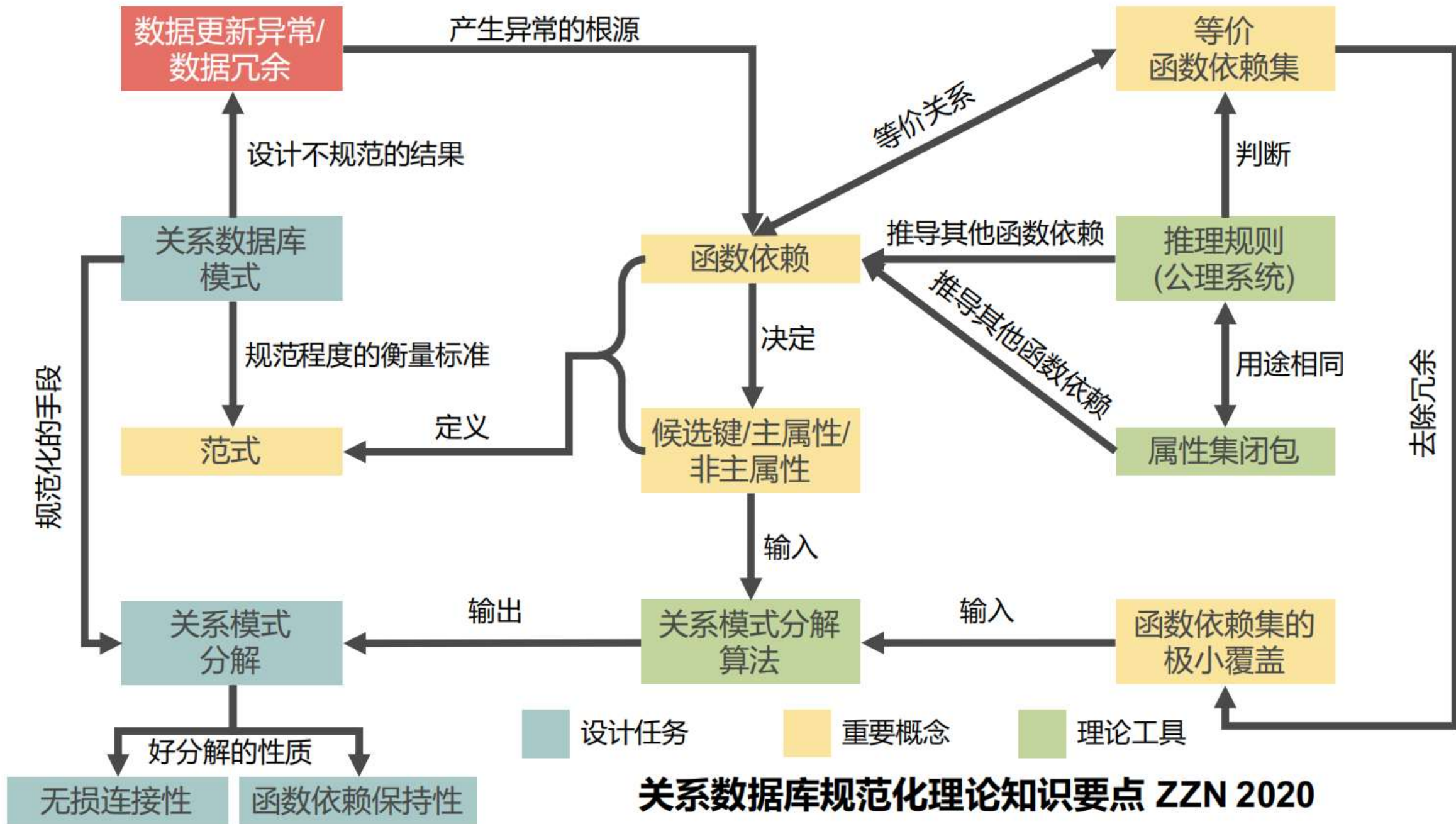
数据库设计过程中应遵循什么样的原则

数据库设计理论

- 数据依赖理论
- 关系范式理论
- 模式分解理论









E-R 转模式



函数依赖



关系范式



模式分解

■ 数据依赖

- 数据依赖(data dependency): 关系的属性在语义上的依赖关系

Example (数据依赖)

关系模式SDC(Sno, Sname, Sdept, Sdean, Cno, Grade)^a的属性间存在如下数据依赖:

- 一个学号(Sno)只对应一个学生姓名(Sname)
- 一名学生(Sno)只属于一个系(Sdept)
- 一个系(Sdept)只有一名系主任(Sdean)
- 一名学生(Sno)只对应一名系主任(Sdean)
- 一名学生学习一门课程(Sno, Cno)只有一个成绩(Grade)

^aSDC代表Student-Department-Course

- 关系模式中的不合理数据依赖会导致一系列问题
 - ▶ 数据更新问题: 数据插入异常、数据删除异常、数据修改繁琐
 - ▶ 数据冗余

■ 问题引入：SDC 关系模式

➤ SDC(Sno, Sname, Sdept, Sdean, Cno, Grade)

Sno	Sname	Sdept	Sdean	Cno	Grade
PH-001	Nick	Physics	Einstein	1002	92
PH-001	Nick	Physics	Einstein	2003	85
PH-001	Nick	Physics	Einstein	3006	88
CS-001	Elsa	CS	Turing	3006	90
CS-001	Elsa	CS	Turing	2003	95
CS-002	Ed	CS	Turing	2003	80
MA-001	Abby	Math	Gauss	1002	100

■ 问题1：数据插入异常

➤ 现象：该插入的数据无法插入

Sno	Sname	Sdept	Sdean	Cno	Grade
PH-001	Nick	Physics	Einstein	1002	92
PH-001	Nick	Physics	Einstein	2003	85
PH-001	Nick	Physics	Einstein	3006	88
CS-001	Elsa	CS	Turing	3006	90
CS-001	Elsa	CS	Turing	2003	95
CS-002	Ed	CS	Turing	2003	80
MA-001	Abby	Math	Gauss	1002	100
		AI	Minsky		

■ 问题2：数据删除异常

➤ 现象：不该删除的数据不得不删除

Sno	Sname	Sdept	Sdean	Cno	Grade
PH-001	Nick	Physics	Einstein	1002	92
PH-001	Nick	Physics	Einstein	2003	85
PH-001	Nick	Physics	Einstein	3006	88
CS-001	Elsa	CS	Turing	3006	90
CS-001	Elsa	CS	Turing	2003	95
CS-002	Ed	CS	Turing	2003	80
MA-001	Abby	Math	Gauss	1002	100

■ 问题3：数据修改繁琐

➤ 现象：数据修改非常繁琐，容易出错

Sno	Sname	Sdept	Sdean	Cno	Grade
PH-001	Nick	Physics	Einstein	1002	92
PH-001	Nick	Physics	Einstein	2003	85
PH-001	Nick	Physics	Einstein	3006	88
CS-001	Elsa	CS	Turing Shannon	3006	90
CS-001	Elsa	CS	Turing Shannon	2003	95
CS-002	Ed	CS	Turing Shannon	2003	80
MA-001	Abby	Math	Gauss	1002	100

■ 问题4：数据冗余

➤ 现象：存在大量冗余数据

Sno	Sname	Sdept	Sdean	Cno	Grade
PH-001	Nick	Physics	Einstein	1002	92
PH-001	Nick	Physics	Einstein	2003	85
PH-001	Nick	Physics	Einstein	3006	88
CS-001	Elsa	CS	Turing	3006	90
CS-001	Elsa	CS	Turing	2003	95
CS-002	Ed	CS	Turing	2003	80
MA-001	Abby	Math	Gauss	1002	100

■ 问题成因及解决方法

- 结论：关系模式 SDC(Sno, Sname, Sdept, Sdean, Cno, Grade)设计的不好
- 原因：由某些不合适的数据依赖导致的

解决方法：

- 运用关系数据库规范化理论（第5.3节）来评估关系模式的规范化程度
- 通过关系模式分解（第5.4节）来消除不合适的数据依赖
- 本节介绍和数据依赖有关的概念和理论

■ 函数依赖的定义

设 $R(U)$ 是属性集 U 上的关系模式, $X \subseteq U, Y \subseteq U$ 。对于 $R(U)$ 的任意关系实例中的任意两个元组 t_1 和 t_2 , 如果由 $t_1[X] = t_2[X]$ 可以推出 $t_1[Y] = t_2[Y]$, 则称 X 函数决定 Y , 或 Y 函数依赖于 X , 记作 $X \rightarrow Y$ 。

关系模式 $SDC(\underline{Sno}, Sname, Sdept, Sdean, \underline{Cno}, Grade)$ 中的函数依赖:

- $Sno \rightarrow Sname$: 一个学号只对应一个学生姓名
- $Sno \rightarrow Sdept$: 一名学生只属于一个系
- $Sdept \rightarrow Sdean$: 一个系只有一名系主任
- $Sno \rightarrow Sdean$: 一名学生只对应一名系主任
- $(Sno, Cno) \rightarrow Grade$: 一名学生学习一门课程只有一个成绩

■ 练习

A	B	C
1	2	3
4	2	3
5	3	3

$A \rightarrow B, A \rightarrow C, B \rightarrow C$

A	B	C	D
a1	b1	c1	d1
a1	b2	c1	d2
a2	b2	c2	d2
a2	b3	c2	d3
a3	b3	c2	d4

$A \rightarrow C, D \rightarrow B$

■ 函数依赖的定义（续）

- 如果 Y 不函数依赖于 X ，则记作 $X \nrightarrow Y$
- 如果 $X \rightarrow Y$ 且 $Y \rightarrow X$ ，则记作 $X \leftrightarrow Y$

注意

- 函数依赖是关系模式的**所有关系实例**上都成立的依赖关系，不能只根据某些关系实例来确定函数依赖
- 函数依赖是语义范畴的概念，只能根据数据的语义来确定函数依赖。例如， $Sname \rightarrow Sno$ 只有在学生不同名时才成立
- 数据库设计者可以对现实世界作出强制规定。例如，规定学生不允许同名，从而使得 $Sname \rightarrow Sno$

■ 函数依赖的分类

- 平凡函数依赖：若 $Y \subseteq X$ ，则 $X \rightarrow Y$ 是平凡函数依赖
- 非平凡函数依赖：若 $Y \not\subseteq X$ ，则 $X \rightarrow Y$ 是非平凡函数依赖
- 完全函数依赖：在关系模式 $R(U)$ 中，如果 $X \rightarrow Y$ ，且对任意 $X' \subset X$ ，都有 $X' \not\rightarrow Y$ ，则称 Y 完全函数依赖于 X ，记作 $X \xrightarrow{f} Y$

举例：关系模型 $SDC(\underline{Sno}, Sname, Sdept, Sdean, \underline{Cno}, Grade)$

- 非平凡函数依赖： $Sno \rightarrow Sname$
- 平凡函数依赖： $(Sno, Sname) \rightarrow Sname$
- 完全函数依赖： $(Sno, Cno) \xrightarrow{f} Grade$ ，因为 $Sno \not\rightarrow Grade$ 且 $Cno \not\rightarrow Grade$

■ 函数依赖的分类 (续)

- 部分函数依赖: 在关系模式 $R(U)$ 中, 如果 $X \rightarrow Y$, 且存在 $X' \subset X$, 使得 $X' \rightarrow Y$, 则称 Y 部分函数依赖于 X , 记作 $X \xrightarrow{p} Y$
- 传递函数依赖: 在关系模式 $R(U)$ 中, 如果 $X \rightarrow Y$, $Y \rightarrow Z$, 且 $Y \not\subseteq X$, $Y \nrightarrow X$, 则称 Z 传递函数依赖于 X , 记作 $X \xrightarrow{t} Z$

举例: 关系模型 $SDC(\underline{Sno}, Sname, Sdept, Sdean, \underline{Cno}, Grade)$

- 部分函数依赖: $(Sno, Cno) \xrightarrow{p} Sname$, 因为 $Sno \rightarrow Sname$
- 传递函数依赖: $Sno \xrightarrow{t} Sdean$, 因为 $Sno \rightarrow Sdept$, $Sdept \nrightarrow Sno$ 且 $Sdept \rightarrow Sdean$

■ 候选键

设 K 为 $R(U)$ 中的属性或属性组合, 若 $K \xrightarrow{f} U$, 则称 K 为 $R(U)$ 上的**候选键**。

包含在任一候选键中的属性称为**主属性**, 其他属性称为**非主属性**。

Sno	Cno	Grade
PH-001	1002	92
PH-001	2003	85
PH-001	3006	88
CS-001	1002	95
CS-001	3006	90
CS-002	3006	80
MA-001	1002	null

以 SC 关系为例:

- $SC(Sno, Cno, Grade)$
- $\{Sno, Cno\} \rightarrow SC$
- $\{Sno, Cno\}$ 为候选键
- Sno, Cno 为主属性
- Grade 为非主属性

■ 函数依赖的公理系统

为什么要学习函数依赖的公理系统？

- 在设计关系模式时，数据库设计者只能给出一部分函数依赖，无法（也没必要）给出全部函数依赖
- 在使用关系数据库规范化理论评估关系模式的规范化程度时，仅知道数据库设计者明确给出函数依赖很可能是不够的
- 使用函数依赖的公理系统，可以在数据库设计者给出函数依赖的基础上推导出其他成立的函数依赖

■ Armstrong 公理系统

Definition (逻辑蕴含)

设 $R(U, F)$ 是一个关系模式，其属性集为 U ，函数依赖集为 F 。若在 R 的任意关系实例 r 中，函数依赖 $X \rightarrow Y$ 都成立(即对于 r 中任意两个元组 t_1 和 t_2 ，若 $t_1[X] = t_2[X]$ ，则 $t_1[Y] = t_2[Y]$)，则称 F 逻辑蕴含 $X \rightarrow Y$ ，记作 $F \models X \rightarrow Y$ 。

Armstrong公理系统

设 $R(U, F)$ 是一个关系模式，Armstrong公理系统包含以下3条推理规则：

- ① 自反律：若 $Y \subseteq X \subseteq U$ ，则 $F \models X \rightarrow Y$
- ② 增广律：若 $F \models X \rightarrow Y$ ，则对任意 $Z \subseteq U$ ，有 $F \models XZ \rightarrow YZ^a$
- ③ 传递律：若 $F \models X \rightarrow Y$ 且 $F \models Y \rightarrow Z$ ，则 $F \models X \rightarrow Z$

^a在关系数据库规范化理论中， XY 代表 $X \cup Y$

■ Armstrong 公理系统 (续)

Theorem

Armstrong公理系统是正确(sound)且完备(complete)的

- 正确性: 使用Armstrong公理系统推出的任何函数依赖一定被 F 逻辑蕴含
- 完备性: 任何被 F 逻辑蕴含的函数依赖一定能够使用Armstrong公理系统推出

Example (Armstrong公理系统)

关系模式SDC(Sno, Sname, Sdept, Sdean, Cno, Grade)上的函数依赖集为 $F = \{Sno \rightarrow Sname, Sno \rightarrow Sdept, Sdept \rightarrow Sdean, (Sno, Cno) \rightarrow Grade\}$, 证明 $F \models (Sno, Cno) \rightarrow (Sdean, Grade)$
(证明过程见黑板)

关系模式 SDC(Sno, Sname, Sdept, Sdean, Cno, Grade)

函数依赖集 $F = \{ Sno \rightarrow Sname, Sno \rightarrow Sdept, Sdept \rightarrow Sdean, (Sno, Cno) \rightarrow Grade \}$

证明: $F \models (Sno, Cno) \rightarrow (Sdean, Grade)$

证:

$\because (Sno, Cno) \rightarrow Grade$

$\therefore (Sno, Cno, Sdean) \rightarrow (Sdean, Grade)$ 增广律 (1)

$\because Sno \rightarrow Sdept, Sdept \rightarrow Sdean$

$\therefore Sno \rightarrow Sdean$ 传递律

$\therefore (Sno, Cno) \rightarrow (Cno, Sdean)$ 增广律

$\therefore (Sno, Cno) \rightarrow (Sno, Cno, Sdean)$ 增广律 (2)

(1) 和 (2) 根据传递律, 得出 $(Sno, Cno) \rightarrow (Sdean, Grade)$

■ Armstrong 公理系统 (续)

Armstrong公理系统的导出规则

设 $R(U, F)$ 是一个关系模式

- ① **合并规则**: 若 $F \models X \rightarrow Y$ 且 $F \models X \rightarrow Z$, 则 $F \models X \rightarrow YZ$
- ② **伪传递规则**: 若 $F \models X \rightarrow Y$ 且 $F \models WY \rightarrow Z$, 则 $F \models XW \rightarrow Z$
- ③ **分解规则**: 若 $F \models X \rightarrow Y$, 则对任意 $Z \subseteq Y$, 有 $F \models X \rightarrow Z$

$\therefore X \rightarrow Y$

$\therefore X \rightarrow XY$ 1)

$\therefore X \rightarrow Z$

$\therefore XY \rightarrow YZ$ 2)

$\therefore 1, 2$ 得出 $X \rightarrow YZ$ (传递律)

$\therefore X \rightarrow Y$

$\therefore WX \rightarrow WY$

$\therefore WY \rightarrow Z$

$\therefore WX \rightarrow Z$ (传递律)

$\therefore Z \subseteq Y$

$\therefore Y \rightarrow Z$ (自反律)

$\therefore X \rightarrow Y$

$\therefore X \rightarrow Z$ (传递律)

关系模式 SDC(Sno, Sname, Sdept, Sdean, Cno, Grade)

函数依赖集 $F = \{ Sno \rightarrow Sname, Sno \rightarrow Sdept, Sdept \rightarrow Sdean, (Sno, Cno) \rightarrow Grade \}$

证明: $F \models (Sno, Cno) \rightarrow (Sdean, Grade)$

证:

$\because Sno \rightarrow Sdept, Sdept \rightarrow Sdean$

$\therefore Sno \rightarrow Sdean$ 传递律 1)

$\because (Sno, Cno) \rightarrow Sno$ 自反律 2)

$\therefore$ 由 1 和 2 得到 $(Sno, Cno) \rightarrow Sdean$ 传递律

$\because (Sno, Cno) \rightarrow Grade$

$\therefore (Sno, Cno) \rightarrow (Sdean, Grade)$ 合并规则

■ 属性集的闭包

为什么要学习属性集的闭包?

- 使用Armstrong公理系统可以证明某函数依赖是否被逻辑蕴含，但是不够方便
- 使用属性集的闭包可以更方便地证明某函数依赖是否被逻辑蕴含

Definition (属性集的闭包)

设 $R(U, F)$ 是一个关系模式，集合 $\{A | A \in U, F \models X \rightarrow A\}$ 称为 X 关于 F 的闭包(closure)，记作 X_F^+

Theorem

设 $R(U, F)$ 是一个关系模式， $X, Y \subseteq U$ ， $F \models X \rightarrow Y$ 的充要条件是 $Y \subseteq X_F^+$

- 逻辑蕴含判定问题 $\Rightarrow$ 属性集闭包计算问题

■ 属性集闭包的计算

算法: 计算属性集的闭包

输入: 属性集 X 、函数依赖集 F

输出: X 关于 F 的闭包 X_F^+

- 1: $i \leftarrow 0; X^{(0)} \leftarrow X$ {初始化}
- 2: **repeat**
- 3: $B \leftarrow \{A \mid V \rightarrow W \in F, V \subseteq X^{(i)}, A \in W\}$ {匹配函数依赖}
- 4: $X^{(i+1)} \leftarrow X^{(i)} \cup B$ {扩充闭包}
- 5: $i \leftarrow i + 1$
- 6: **until** $X^{(i)} = X^{(i-1)}$ 或 $X^{(i)} = U$ {终止条件}
- 7: **return** $X^{(i)}$

Example (属性集的闭包)

关系模式SDC(Sno, Sname, Sdept, Sdean, Cno, Grade)上的函数依赖集为 $F = \{Sno \rightarrow Sname, Sno \rightarrow Sdept, Sdept \rightarrow Sdean, (Sno, Cno) \rightarrow Grade\}$, 证明 $F \models (Sno, Cno) \rightarrow (Sdean, Grade)$

■ 属性集闭包的计算 (续)

Example (属性集的闭包)

① 计算 $(Sno, Cno)_F^+$

i	匹配的函数依赖	集合 $X^{(i)}$
0		(Sno, Cno)
1	$Sno \rightarrow Sname$ $Sno \rightarrow Sdept$ $Sdept \rightarrow Sdean$ $(Sno, Cno) \rightarrow Grade$	$(Sno, Cno, Sname, Sdept, Grade)$
2	$Sno \rightarrow Sname$ $Sno \rightarrow Sdept$ $Sdept \rightarrow Sdean$ $(Sno, Cno) \rightarrow Grade$	$(Sno, Cno, Sname, Sdept, Grade, Sdean)$ $(X^{(i)} = U, \text{终止})$

② 因为 $(Sdean, Grade) \subseteq (Sno, Cno)_F^+$, 所以 $F \models (Sno, Cno) \rightarrow (Sdean, Grade)$

■ 等价函数依赖集

为什么要学习等价函数依赖集？

- 在设计关系模式时，不同设计者会给出（表面上）不同的函数依赖集
- 如果这些表面上不同的函数依赖集能够逻辑蕴含相同的函数依赖，那么它们本质上是一样的，因此是等价的
- 然而，等价的函数依赖集的简练程度却未必相同，我们希望从全部等价的函数依赖集中找出最简练的那个

■ 等价函数依赖集的相关概念

Definition (函数依赖集的闭包)

设 $R(U, F)$ 是一个关系模式，由 F 逻辑蕴含的全部函数依赖的集合叫做 F 的闭包(closure)，记作 F^+

Definition (等价函数依赖集)

设 F 和 G 是关系模式 $R(U)$ 上的两个函数依赖集，如果 $F^+ = G^+$ ，则 F 与 G 等价

Definition (函数依赖集的覆盖)

设 F 和 G 是关系模式 $R(U)$ 上的两个函数依赖集，如果 $G^+ \subseteq F^+$ (即 G 中每个函数依赖都被 F 逻辑蕴含)，则称 F 覆盖 G (F covers G)

Theorem

函数依赖集 F 与 G 等价，当且仅当 F 覆盖 G 且 G 覆盖 F

■ 等价函数依赖集的判定

- 方法1: 使用 $F^+ = G^+$ 来判定 F 与 G 等价, 但是计算 F^+ 和 G^+ 的代价太高
- 方法2: 如果 F 覆盖 G (F 逻辑蕴含 G 中任意函数依赖), 且 G 覆盖 F (G 逻辑蕴含 F 中任意函数依赖), 则 F 与 G 等价

Example (等价函数依赖集)

判断 F 和 G 是否等价:

- $F = \{Sno \rightarrow Sname, Sno \rightarrow Sdept, Sdept \rightarrow Sdean, (Sno, Cno) \rightarrow Grade\}$
- $G = \{Sno \rightarrow Sname, Sno \rightarrow Sdept, Sno \rightarrow Sdean, (Sno, Cno) \rightarrow Grade\}$

我们可以使用 Armstrong 公理或属性集闭包证明

- F 逻辑蕴含 G 中所有函数依赖
- $G \not\models Sdept \rightarrow Sdean$

所以 F 和 G 不等价

■ 最小覆盖

Definition (最小覆盖)

关系模式 $R(U)$ 的函数依赖集 F 的最小覆盖(canonical cover)是满足下列3个条件的 F 的等价函数依赖集 F_c

- ① (不存在冗余函数依赖) F_c 中不存在函数依赖 $X \rightarrow Y$, 使得 F_c 与 $F_c - \{X \rightarrow Y\}$ 等价
- ② (函数依赖左部不存在冗余属性) F_c 中不存在函数依赖 $X \rightarrow Y$, 有 $Z \subset X$ 使得 F_c 与 $(F_c - \{X \rightarrow Y\}) \cup \{Z \rightarrow Y\}$ 等价
- ③ (函数依赖左部唯一) F_c 中任意两个函数依赖的左部不相同^a

^a有些教材中使用另一种条件: F_c 中任意函数依赖的右部只包含1个属性

■ 最小覆盖的计算

最小覆盖的计算算法

输入: 函数依赖集 F

输出: F 的最小覆盖

1: **repeat**

2: (删除冗余函数依赖) 对 F 中任意函数依赖 $X \rightarrow Y$,
 若 F 与 $F - \{X \rightarrow Y\}$ 等价, 则将 $\{X \rightarrow Y\}$ 从 F 中删除

3: (删除函数依赖左部的冗余属性) 对 F 中任意函数依赖 $X \rightarrow Y$, 若
 存在 $A \in X$, 使得 F 与 $F - \{X \rightarrow Y\} \cup \{X - \{A\} \rightarrow Y\}$ 等价, 则
 将 $X \rightarrow Y$ 替换为 $X - \{A\} \rightarrow Y$

4: (合并函数依赖) 使用合并规则, 将 F 中左部相同的两个函数依
 赖 $X \rightarrow Y$ 和 $X \rightarrow Z$ 合并为一个函数依赖 $X \rightarrow YZ$

5: **until** F 不再变化

6: **return** F



E-R 转模式



函数依赖



关系范式



模式分解

■ 关系模式的范式

- 范式：在关系数据库规范化理论中，根据关系模式的规范化程度不同，把关系模式划分为若干类，称作范式（normal form）
- 范式的级别
 - 第一范式（1NF）
 - 第二范式（2NF）
 - 第三范式（3NF）
 - Boyce-Codd范式（BCNF）
 - 第四范式（4NF）
 - 第五范式（5NF）
- 范式的关系： $5NF \subseteq 4NF \subseteq BCNF \subseteq 3NF \subseteq 2NF \subseteq 1NF$
- 关系模式的规范化程度越高，数据更新导致的问题越少
- 在函数依赖的范畴内，我们只讨论 1NF、2NF、3NF 和 BCNF

■ 第一范式 (1NF)

如果关系模式 R 的每个属性都是不可分的, 则称 R 为第一范式关系模式, 记作 $R \in 1NF$

举例: 关系模型 $SDC(\underline{Sno}, Sname, Sdept, Sdean, \underline{Cno}, Grade)$

➤ 函数依赖

$Sno \rightarrow Sname, Sno \rightarrow Sdept, Sdept \rightarrow Sdean, (Sno, Cno) \rightarrow Grade$

➤ 结论

$SDC \in 1NF$

(Sno, Cno) 为 SDC 的候选键, $(Sno, Cno) \xrightarrow{p} Sname$

■ 1NF 关系模式存在的问题：数据插入异常

➤ 现象：该插入的数据无法插入

Sno	Sname	Sdept	Sdean	Cno	Grade
PH-001	Nick	Physics	Einstein	1002	92
PH-001	Nick	Physics	Einstein	2003	85
PH-001	Nick	Physics	Einstein	3006	88
CS-001	Elsa	CS	Turing	3006	90
CS-001	Elsa	CS	Turing	2003	95
CS-002	Ed	CS	Turing	2003	80
MA-001	Abby	Math	Gauss	1002	100
MA-002	Cindy	Math	Gauss		

■ 1NF 关系模式存在的问题：数据删除异常

➤ 现象：不该删除的数据不得不删除

Sno	Sname	Sdept	Sdean	Cno	Grade
PH-001	Nick	Physics	Einstein	1002	92
PH-001	Nick	Physics	Einstein	2003	85
PH-001	Nick	Physics	Einstein	3006	88
CS-001	Elsa	CS	Turing	3006	90
CS-001	Elsa	CS	Turing	2003	95
CS-002	Ed	CS	Turing	2003	80
MA-001	Abby	Math	Gauss	1002	100

■ 1NF 关系模式存在的问题：数据修改繁琐

➤ 现象：数据修改非常繁琐，容易出错

Sno	Sname	Sdept	Sdean	Cno	Grade
PH-001	Nick	Physics CS	Einstein Turing	1002	92
PH-001	Nick	Physics CS	Einstein Turing	2003	85
PH-001	Nick	Physics CS	Einstein Turing	3006	88
CS-001	Elsa	CS	Turing	3006	90
CS-001	Elsa	CS	Turing	2003	95
CS-002	Ed	CS	Turing	2003	80
MA-001	Abby	Math	Gauss	1002	100

■ 1NF 关系模式存在的问题：数据冗余

➤ 现象：存在大量冗余数据

Sno	Sname	Sdept	Sdean	Cno	Grade
PH-001	Nick	Physics	Einstein	1002	92
PH-001	Nick	Physics	Einstein	2003	85
PH-001	Nick	Physics	Einstein	3006	88
CS-001	Elsa	CS	Turing	3006	90
CS-001	Elsa	CS	Turing	2003	95
CS-002	Ed	CS	Turing	2003	80
MA-001	Abby	Math	Gauss	1002	100

■ 1NF 关系模式问题的解决方法

- 问题原因：非主属性 Sdept 和 Sdean 部分函数依赖候选键 (Sno, Cno)
- 解决办法：把 SDC 分解为 SD(Sno, Sname, Sdept, Sdean) 和 SC(Sno, Cno, Grade)

Sno	Sname	Sdept	Sdean
PH-001	Nick	Physics	Einstein
CS-001	Elsa	CS	Turing
CS-002	Ed	CS	Turing
MA-001	Abby	Math	Gauss

Sno	Cno	Grade
PH-001	1002	92
PH-001	2003	85
PH-001	3006	88
CS-001	3006	90
CS-001	2003	95
CS-002	2003	80
MA-001	1002	100

■ 1NF 关系模式问题的解决方法

➤ 解决了数据插入异常

Sno	Sname	Sdept	Sdean
PH-001	Nick	Physics	Einstein
CS-001	Elsa	CS	Turing
CS-002	Ed	CS	Turing
MA-001	Abby	Math	Gauss
MA-002	Cindy	Math	Gauss

Sno	Cno	Grade
PH-001	1002	92
PH-001	2003	85
PH-001	3006	88
CS-001	3006	90
CS-001	2003	95
CS-002	2003	80
MA-001	1002	100

■ 1NF 关系模式问题的解决方法

➤ 解决了数据删除异常

Sno	Sname	Sdept	Sdean
PH-001	Nick	Physics	Einstein
CS-001	Elsa	CS	Turing
CS-002	Ed	CS	Turing
MA-001	Abby	Math	Gauss

Sno	Cno	Grade
PH-001	1002	92
PH-001	2003	85
PH-001	3006	88
CS-001	3006	90
CS-001	2003	95
CS-002	2003	80
MA-001	1002	100

■ 1NF 关系模式问题的解决方法

➤ 简化了数据修改

Sno	Sname	Sdept	Sdean
PH-001	Nick	Physics CS	Einstein Turing
CS-001	Elsa	CS	Turing
CS-002	Ed	CS	Turing
MA-001	Abby	Math	Gauss

Sno	Cno	Grade
PH-001	1002	92
PH-001	2003	85
PH-001	3006	88
CS-001	3006	90
CS-001	2003	95
CS-002	2003	80
MA-001	1002	100

■ 1NF 关系模式问题的解决方法

➤ 减少了数据冗余

Sno	Sname	Sdept	Sdean
PH-001	Nick	Physics	Einstein
CS-001	Elsa	CS	Turing
CS-002	Ed	CS	Turing
MA-001	Abby	Math	Gauss

Sno	Cno	Grade
PH-001	1002	92
PH-001	2003	85
PH-001	3006	88
CS-001	3006	90
CS-001	2003	95
CS-002	2003	80
MA-001	1002	100

■ 第二范式 (2NF)

如果关系模式 $R \in 1NF$ ，且 R 的每个非主属性都完全函数依赖于 R 的候选键，则称 R 为第二范式关系模式，记作 $R \in 2NF$

举例：

➤ $SDC(\underline{Sno}, Sname, Sdept, Sdean, \underline{Cno}, Grade) \notin 2NF$

原因： $SDC \in 1NF$ ，但 $(Sno, Cno) \xrightarrow{p} Sname$

➤ $SD(\underline{Sno}, Sname, Sdept, Sdean) \in 2NF$

原因： $SD \in 1NF$ ，且 $Sname, Sdept, Sdean$ 均完全函数依赖 Sno

➤ $SC(\underline{Sno}, \underline{Cno}, Grade) \in 2NF$

原因： $SC \in 1NF$ ，且 $Grade$ 完全函数依赖 (Sno, Cno)

■ 2NF 关系模式存在的问题：数据插入异常

➤ 现象：该插入的数据无法插入

Sno	Sname	Sdept	Sdean
PH-001	Nick	Physics	Einstein
CS-001	Elsa	CS	Turing
CS-002	Ed	CS	Turing
MA-001	Abby	Math	Gauss
		AI	Minsky

■ 2NF 关系模式存在的问题：数据删除异常

➤ 现象：不该删除的数据不得不删除

Sno	Sname	Sdept	Sdean
PH-001	Nick	Physics	Einstein
CS-001	Elsa	CS	Turing
CS-002	Ed	CS	Turing
MA-001	Abby	Math	Gauss

■ 2NF 关系模式存在的问题：数据修改繁琐

➤ 现象：数据修改非常繁琐，容易出错

Sno	Sname	Sdept	Sdean
PH-001	Nick	Physics	Einstein
CS-001	Elsa	CS	Turing Shannon
CS-002	Ed	CS	Turing Shannon
MA-001	Abby	Math	Gauss

■ 2NF 关系模式存在的问题：数据冗余

➤ 现象：存在大量冗余数据

Sno	Sname	Sdept	Sdean
PH-001	Nick	Physics	Einstein
CS-001	Elsa	CS	Turing
CS-002	Ed	CS	Turing
MA-001	Abby	Math	Gauss

■ 2NF 关系模式问题的解决方法

- 问题原因：非主属性 Sdean 传递函数依赖候选键 Sno
- 解决办法：把 SD 分解为 S(Sno, Sname, Sdept) 和 D(Sdept, Sdean)

Sno	Sname	Sdept
PH-001	Nick	Physics
CS-001	Elsa	CS
CS-002	Ed	CS
MA-001	Abby	Math

Sdept	Sdean
Physics	Einstein
CS	Turing
Math	Gauss

■ 2NF 关系模式问题的解决方法

➤ 解决了数据插入异常

Sno	Sname	Sdept
PH-001	Nick	Physics
CS-001	Elsa	CS
CS-002	Ed	CS
MA-001	Abby	Math

Sdept	Sdean
Physics	Einstein
CS	Turing
Math	Gauss
AI	Minsky

■ 2NF 关系模式问题的解决方法

➤ 解决了数据删除异常

Sno	Sname	Sdept
PH-001	Nick	Physics
CS-001	Elsa	CS
CS-002	Ed	CS
MA-001	Abby	Math

Sdept	Sdean
Physics	Einstein
CS	Turing
Math	Gauss

■ 2NF 关系模式问题的解决方法

➤ 简化了数据修改

Sno	Sname	Sdept
PH-001	Nick	Physics
CS-001	Elsa	CS
CS-002	Ed	CS
MA-001	Abby	Math

Sdept	Sdean
Physics	Einstein
CS	Turing Shannon
Math	Gauss

■ 2NF 关系模式问题的解决方法

➤ 减少了数据冗余

Sno	Sname	Sdept
PH-001	Nick	Physics
CS-001	Elsa	CS
CS-002	Ed	CS
MA-001	Abby	Math

Sdept	Sdean
Physics	Einstein
CS	Turing
Math	Gauss

■ 第三范式 (3NF)

如果关系模式 $R \in 2NF$ ，且 R 的每个非主属性都不传递函数依赖于 R 的候选键，则称 R 为第三范式关系模式，记作 $R \in 3NF$

举例：

➤ $SD(\underline{Sno}, Sname, Sdept, Sdean) \notin 3NF$

原因： $SD \in 2NF$ ，但非主属性 $Sdean$ 传递函数依赖于 SD 的候选键 Sno

➤ $S(\underline{Sno}, Sname, Sdept) \in 3NF$

原因： $S \in 2NF$ ，且非主属性 $Sname$ 和 $Sdept$ 均直接函数依赖于 S 的候选键 Sno

➤ $D(\underline{Sdept}, Sdean) \in 3NF$

原因： $D \in 2NF$ ，且非主属性 $Sdean$ 直接函数依赖于 D 的候选键 $Sdept$

■ 3NF 关系模式存在的问题

关系模式 $STC(Sno, Tno, Cno)$, Sno 为学号, Tno 为教师号, Cno 为课程号

举例:

➤ $STC(Sno, Tno, Cno) \in 3NF$

$(Sno, Cno) \rightarrow Tno$

$Tno \rightarrow Cno$ (假设一名教师只教一门课)

➤ 结论

(Sno, Cno) 和 (Sno, Tno) 都是 STC 的候选键

$STC \in 3NF$, 因为不存在非主属性对候选键的

传递函数依赖 (STC 中根本没有非主属性)

Sno	Tno	Cno
S1	T1	C1
S2	T1	C1
S3	T2	C2
S4	T2	C2
S5	T3	C2

■ 3NF 关系模式存在的问题：数据插入异常

➤ 现象：该插入的数据无法插入

Sno	Tno	Cno
S1	T1	C1
S2	T1	C1
S3	T2	C2
S4	T2	C2
S5	T3	C2
	T4	C3

■ 3NF 关系模式存在的问题：数据删除异常

➤ 现象：不该删除的数据不得不删除

Sno	Tno	Cno
S1	T1	C1
S2	T1	C1
S3	T2	C2
S4	T2	C2
S5	T3	C2

■ 3NF 关系模式存在的问题：数据修改繁琐

➤ 现象：数据修改非常繁琐，容易出错

Sno	Tno	Cno
S1	T1	C1 C4
S2	T1	C1 C4
S3	T2	C2
S4	T2	C2
S5	T3	C2

■ 3NF 关系模式存在的问题：数据冗余

➤ 现象：存在大量冗余数据

Sno	Tno	Cno
S1	T1	C1
S2	T1	C1
S3	T2	C2
S4	T2	C2
S5	T3	C2

■ 3NF 关系模式问题的解决方法

- 问题原因：主属性 Cno 部分函数依赖候选键 (Sno, Tno)
- 解决办法：把 STC 分解为 TC(Tno, Cno) 和 ST(Sno, Tno)

Tno	Cno
T1	C1
T2	C2
T3	C2

Sno	Tno
S1	T1
S2	T1
S3	T2
S4	T2
S5	T3

■ Boyce-Codd 范式 (BCNF)

若关系模式 $R(U, F) \in 3NF$, 且 R 的任意主属性都直接完全函数依赖于 R 的候选键, 则称 R 为 Boyce-Codd 范式关系模式, 记作 $R \in BCNF$

举例:

➤ $STC(\underline{Sno}, Tno, Cno) \notin BCNF$

原因: $STC \in 3NF$, 但 $Tno \rightarrow Cno$ 的左部不是候选键

➤ $TC(\underline{Tno}, Cno) \in BCNF$

原因: $TC \in 3NF$, 且 $Tno \rightarrow Cno$ 的左部是候选键

➤ $ST(\underline{Sno}, Tno) \in BCNF$

原因: $ST \in 3NF$, 候选键是 (Sno, Tno)

■ 总结

1NF 属性不可再分



2NF 消除非主属性对候选键的部分函数依赖



3NF 消除非主属性对候选键的传递函数依赖



BCNF 消除主属性对候选键的部分函数依赖和传递函数依赖

在函数依赖范畴内，BCNF 已经达到了最高的范式级别

■ 从 E-R 到规范的关系模式

1NF SDC(Sno, Sname, Sdept, Sdean, Cno, Grade)

↓

2NF SD(Sno, Sname, Sdept, Sdean) 和 SC(Sno, Cno, Grade)

↓

3NF S(Sno, Sname, Sdept)、D(Sdept, Sdean) 和 SC(Sno, Cno, Grade)

↓

BCNF ...

从 E-R 模型转成关系模式，本身就可以设计出规范化的关系模式



E-R 转模式



函数依赖



关系范式



模式分解

■ 关系模式分解

关系模式规范化是通过关系模式分解实现的

Definition (关系模式分解)

设 $R(U)$ 是属性集 U 上的关系模式， $\rho = \{R_1(U_1), R_2(U_2), \dots, R_n(U_n)\}$ 是 R 的一个分解(decomposition)，如果 $U = U_1 \cup U_2 \cup \dots \cup U_n$ ，且对任意 $i \neq j$ ，有 $U_i \not\subseteq U_j$ (无冗余)

Example (关系模式分解)

$\rho = \{S(Sno, Sname, Sdept), D(Sdept, Sdean), SC(Sno, Cno, Grade)\}$ 是对关系模式 $SDC(Sno, Sname, Sdept, Sdean, Cno, Grade)$ 的一个分解

- 关系模式分解的方法是不唯一的
- 在各种分解方法中，只有能够保证分解后的关系模式与原关系模式等价的分解方法才有意义

■ 关系模式分解 (续)

关系模式SDN(Sno, Sdept, Sdean)有多种分解方法

Example (分解方法1)

将SDN分解为S(Sno), D(Sdept), N(Sdean)

- $S, D, N \in BCNF$ (规范)
- $SDN \neq S \bowtie D \bowtie N$ (连接有损)

SDN					
Sno	Sdept	Sdean			
PH-001	Physics	Einstein	$\Rightarrow$	S	
CS-001	CS	Turing		Sno	
CS-002	CS	Turing		PH-001	
MA-001	Math	Gauss		CS-001	
241101	AI	Turing		CS-002	
				MA-001	
				241101	
				D	
				Sdept	
				Physics	
				CS	
				Math	
				AI	
				N	
				Sdean	
				Einstein	
				Turing	
				Gauss	

■ 关系模式分解 (续)

Example (分解方法2)

将SDN分解为SN(Sno, Sdean), DN(Sdept, Sdean)

- $SN, DN \in BCNF$ (规范)
- $SDN \neq SN \bowtie DN$ (连接有损)

SDN			SN		DN	
Sno	Sdept	Sdean	Sno	Sdean	Sdept	Sdean
PH-001	Physics	Einstein	PH-001	Einstein	Physics	Einstein
CS-001	CS	Turing	CS-001	Turing	CS	Turing
CS-002	CS	Turing	CS-002	Turing	Math	Gauss
MA-001	Math	Gauss	MA-001	Gauss	AI	Turing
241101	AI	Turing	241101	Turing		

■ 关系模式分解 (续)

Example (分解方法3)

将SDN分解为SD(Sno, Sdept), SN(Sno, Sdean)

- $SD, SN \in BCNF$ (规范)
- $SDN = SD \bowtie SN$ (连接无损)
- $Sno \rightarrow Sdept$ 保留下来, 但 $Sdept \rightarrow Sdean$ 不存在了 (未保持函数依赖)

SDN			SD		SN	
Sno	Sdept	Sdean	Sno	Sdept	Sno	Sdean
PH-001	Physics	Einstein	PH-001	Physics	PH-001	Einstein
CS-001	CS	Turing	CS-001	CS	CS-001	Turing
CS-002	CS	Turing	CS-002	CS	CS-002	Turing
MA-001	Math	Gauss	MA-001	Math	MA-001	Gauss
241101	AI	Turing	241101	AI	241101	Turing

■ 关系模式分解 (续)

Example (分解方法4)

将SDN分解为SD(Sno, Sdept), DN(Sdept, Sdean)

- $SD, DN \in BCNF$ (规范)
- $SDN = SD \bowtie DN$ (连接无损)
- $Sno \rightarrow Sdept$ 和 $Sdept \rightarrow Sdean$ 都保留下来了 (保持函数依赖)

SDN			SD		DN	
Sno	Sdept	Sdean	Sno	Sdept	Sdept	Sdean
PH-001	Physics	Einstein	PH-001	Physics	Physics	Einstein
CS-001	CS	Turing	CS-001	CS	CS	Turing
CS-002	CS	Turing	CS-002	CS		
MA-001	Math	Gauss	MA-001	Math	Math	Gauss
241101	AI	Turing	241101	AI	AI	Turing

■ 关系模式分解准则 1：无损连接性

Definition (无损连接分解)

设 $\rho = \{R_1(U_1), R_2(U_2), \dots, R_n(U_n)\}$ 是对关系模式 $R(U)$ 的一个分解，如果对于 R 的任意实例 r ，都有 $r = \Pi_{U_1}(r) \bowtie \Pi_{U_2}(r) \bowtie \dots \bowtie \Pi_{U_n}(r)$ ，则称 ρ 是 R 的无损连接分解

Example (无损连接分解)

$\{SD(Sno, Sdept), DN(Sdept, Sdean)\}$ 是对 SDN 的无损连接分解

SDN			SD		DN	
Sno	Sdept	Sdean	Sno	Sdept	Sdept	Sdean
PH-001	Physics	Einstein	PH-001	Physics	Physics	Einstein
CS-001	CS	Turing	CS-001	CS	CS	Turing
CS-002	CS	Turing	CS-002	CS	Math	Gauss
MA-001	Math	Gauss	MA-001	Math	AI	Turing
241101	AI	Turing	241101	AI		

■ 无损连接分解可能存在的问题

对关系模式进行无损连接分解后，可能还存在数据更新异常、数据冗余等问题

Example (无损连接分解的问题)

- $\{SD(Sno, Sdept), SN(Sno, Sdean)\}$ 是对SDN的无损连接分解
- 操作: PH-001号学生从物理系(Physics)转到计算机系(CS)
- 结果: 既要修改该学生在ST中的Sdept属性值，也要修改该学生在SN中的Sdean属性值，否则会破坏数据一致性
- 原因: SDN上原有的函数依赖 $Sdept \rightarrow Sdean$ 在分解后丢失了

SD	
Sno	Sdept
PH-001	Physics CS
CS-001	CS
CS-002	CS
MA-001	Math
241101	AI

SN	
Sno	Sdean
PH-001	Einstein Turing
CS-001	Turing
CS-002	Turing
MA-001	Gauss
241101	Turing

■ 无损连接分解可能存在的问题 (续)

Example (无损连接分解的问题)

- $\{SD(Sno, Sdept), DN(Sdept, Sdean)\}$ 是对SDN的无损连接分解
- 操作: PH-001号学生从物理系(Physics)转到计算机系(CS)
- 结果: 只需修改该学生在ST中的Sdept属性值
- 原因: SDN上原有的函数依赖 $Sdept \rightarrow Sdean$ 在DN中保留了

SD	
Sno	Sdept
PH-001	Physics CS
CS-001	CS
CS-002	CS
MA-001	Math
241101	AI

DN	
Sdept	Sdean
Physics	Einstein
CS	Turing
Math	Gauss
AI	Turing

■ 关系模式分解准则2：函数依赖保持

Definition (函数依赖集的投影)

设 $R(U, F)$ 是关系模式， U 是属性集， F 是函数依赖集。对于 $V \subseteq U$ ， F 在 V 上的投影是 F^+ 中满足 $X \cup Y \subseteq V$ 的函数依赖 $X \rightarrow Y$ 的集合

Example (函数依赖集的投影)

SDN上的函数依赖集 $F = \{Sno \rightarrow Sdept, Sdept \rightarrow Sdean\}$

- F 在 $(Sno, Sdept)$ 上的投影是 $\{Sno \rightarrow Sdept\}$
- F 在 $(Sno, Sdean)$ 上的投影是 $\{Sno \rightarrow Sdean\}$ (并未直接出现在 F 中)
- F 在 $(Sdept, Sdean)$ 上的投影是 $\{Sdept \rightarrow Sdean\}$

Definition (保持函数依赖分解)

设 $R(U, F)$ 是一个关系模式， U 是属性集， F 是函数依赖集。

设 $\rho = \{R_1(U_1), R_2(U_2), \dots, R_n(U_n)\}$ 是对 R 的一个分解， F_i 是 F 在 U_i 上的投影。如果 $F_1 \cup F_2 \cup \dots \cup F_n$ 覆盖 F ，则 ρ 是保持函数依赖的模式分解。

■ 关系模式分解准则2：函数依赖保持（续）

Example (保持函数依赖的分解)

SDN上的函数依赖集为 $F = \{Sno \rightarrow Sdept, Sdept \rightarrow Sdean\}$ 。
将SDN分解为SD(Sno, Sdept), SN(Sno, Sdean)

- F 在SD(Sno, Sdept)上的投影为 $F_{SD} = \{Sno \rightarrow Sdept\}$
- F 在SN(Sno, Sdean)上的投影为 $F_{SN} = \{Sno \rightarrow Sdean\}$
- $F_{SD} \cup F_{SN}$ 不能覆盖 F (如何证明?)
- 该分解不是保持函数依赖的分解

Example (保持函数依赖的分解)

将SDN分解为SD(Sno, Sdept), DN(Sdept, Sdean)

- F 在SD(Sno, Sdept)上的投影为 $F_{SD} = \{Sno \rightarrow Sdept\}$
- F 在DN(Sdept, Sdean)上的投影为 $F_{DN} = \{Sdept \rightarrow Sdean\}$
- $F_{SD} \cup F_{DN} = F$, 一定能覆盖 F
- 该分解是保持函数依赖的分解

■ 关系模式分解准则（续）

- 一个好的关系模式分解需要满足的特性
 - 无损连接性：确保分解后数据不丢失
 - 函数依赖保持性：减轻或解决异常情况
- 无损连接性和函数依赖保持性是两个相互独立的准则
 - 无损连接分解不一定是保持函数依赖的分解
 - 保持函数依赖的分解也不一定是无损连接分解
- 如何判断一个关系模式分解是否满足无损连接性？
- 如何判断一个关系模式分解是否满足函数依赖保持性？

■ 无损连接性的判定

- 目的: 判定一个关系模式分解是否满足无损连接性
- 方法1 (一个关系模式被分解为两个关系模式时的判定方法)

Theorem

关系模式 $R(U, F)$ 的分解 $\rho = \{R_1(U_1), R_2(U_2)\}$ 满足无损连接性的充要条件是 $F \models U_1 \cap U_2 \rightarrow U_1$ 或 $F \models U_1 \cap U_2 \rightarrow U_2$

Example (无损连接性的判定1)

SDN上的函数依赖集为 $F = \{Sno \rightarrow Sdept, Sdept \rightarrow Sdean\}$ 。
将SDN分解为SD(Sno, Sdept), SN(Sno, Sdean)

- $Sno \rightarrow (Sno, Sdept)$
- $Sno \rightarrow (Sno, Sdean)$
- 该分解是无损连接分解

■ 无损连接性的判定 (续)

Example (无损连接性的判定2)

将SDN分解为SD(Sno, Sdept), DN(Sdept, Sdean)

- $Sdept \twoheadrightarrow (Sno, Sdept)$
- $Sdept \rightarrow (Sdept, Sdean)$
- 该分解是无损连接分解

Example (无损连接性的判定3)

将SDN分解为SN(Sno, Sdean), DN(Sdept, Sdean)

- $Sdean \twoheadrightarrow (Sno, Sdean)$
- $Sdean \twoheadrightarrow (Sdept, Sdean)$
- 该分解不是无损连接分解

■ 无损连接性的判定 (续)

- 目的: 判定一个关系模式分解是否满足无损连接性
- 方法2 (一个关系模式被分解为3个以上关系模式时的判定方法)

Example

- 属性集 $U = \{A, B, C, D, E\}$
- 函数依赖集 $F = \{A \rightarrow C, B \rightarrow C, C \rightarrow D, DE \rightarrow C, CE \rightarrow A\}$
- 关系模式分解 $\rho = \{(A, D), (A, B), (B, E), (C, D, E), (A, E)\}$

1. 初始化

	A	B	C	D	E
$R_1(A, D)$	a	$x_{1,2}$	$x_{1,3}$	d	$x_{1,5}$
$R_2(A, B)$	a	b	$x_{2,3}$	$x_{2,4}$	$x_{2,5}$
$R_3(B, E)$	$x_{3,1}$	b	$x_{3,3}$	$x_{3,4}$	e
$R_4(C, D, E)$	$x_{4,1}$	$x_{4,2}$	c	d	e
$R_5(A, E)$	a	$x_{5,2}$	$x_{5,3}$	$x_{5,4}$	e

■ 无损连接性的判定 (续)

Example

- 函数依赖集 $F = \{A \rightarrow C, B \rightarrow C, C \rightarrow D, DE \rightarrow C, CE \rightarrow A\}$

2. 根据 $A \rightarrow C$, 可知 $x_{1,3} = x_{2,3} = x_{5,3}$

	A	B	C	D	E
$R_1(A, D)$	a	$x_{1,2}$	$x_{1,3}$	d	$x_{1,5}$
$R_2(A, B)$	a	b	$x_{2,3} = x_{1,3}$	$x_{2,4}$	$x_{2,5}$
$R_3(B, E)$	$x_{3,1}$	b	$x_{3,3}$	$x_{3,4}$	e
$R_4(C, D, E)$	$x_{4,1}$	$x_{4,2}$	c	d	e
$R_5(A, E)$	a	$x_{5,2}$	$x_{5,3} = x_{1,3}$	$x_{5,4}$	e

3. 根据 $B \rightarrow C$, 可知 $x_{1,3} = x_{3,3}$

	A	B	C	D	E
$R_1(A, D)$	a	$x_{1,2}$	$x_{1,3}$	d	$x_{1,5}$
$R_2(A, B)$	a	b	$x_{1,3}$	$x_{2,4}$	$x_{2,5}$
$R_3(B, E)$	$x_{3,1}$	b	$x_{3,3} = x_{1,3}$	$x_{3,4}$	e
$R_4(C, D, E)$	$x_{4,1}$	$x_{4,2}$	c	d	e
$R_5(A, E)$	a	$x_{5,2}$	$x_{1,3}$	$x_{5,4}$	e

■ 无损连接性的判定 (续)

Example

- 函数依赖集 $F = \{A \rightarrow C, B \rightarrow C, C \rightarrow D, DE \rightarrow C, CE \rightarrow A\}$

4. 根据 $C \rightarrow D$, 可知 $x_{2,4} = x_{3,4} = x_{5,4} = \mathbf{d}$

	A	B	C	D	E
$R_1(A, D)$	a	$x_{1,2}$	$x_{1,3}$	d	$x_{1,5}$
$R_2(A, B)$	a	b	$x_{1,3}$	$x_{2,4} = \mathbf{d}$	$x_{2,5}$
$R_3(B, E)$	$x_{3,1}$	b	$x_{1,3}$	$x_{3,4} = \mathbf{d}$	e
$R_4(C, D, E)$	$x_{4,1}$	$x_{4,2}$	c	d	e
$R_5(A, E)$	a	$x_{5,2}$	$x_{1,3}$	$x_{5,4} = \mathbf{d}$	e

5. 根据 $DE \rightarrow C$, 可知 $x_{1,3} = \mathbf{c}$

	A	B	C	D	E
$R_1(A, D)$	a	$x_{1,2}$	$x_{1,3} = \mathbf{c}$	d	$x_{1,5}$
$R_2(A, B)$	a	b	$x_{1,3} = \mathbf{c}$	d	$x_{2,5}$
$R_3(B, E)$	$x_{3,1}$	b	$x_{1,3} = \mathbf{c}$	d	e
$R_4(C, D, E)$	$x_{4,1}$	$x_{4,2}$	c	d	e
$R_5(A, E)$	a	$x_{5,2}$	$x_{1,3} = \mathbf{c}$	d	e

■ 无损连接性的判定 (续)

Example

- 函数依赖集 $F = \{A \rightarrow C, B \rightarrow C, C \rightarrow D, DE \rightarrow C, CE \rightarrow A\}$

6. 根据 $CE \rightarrow A$, 可知 $x_{3,1} = x_{4,1} = \mathbf{a}$

	A	B	C	D	E
$R_1(A, D)$	a	$x_{1,2}$	c	d	$x_{1,5}$
$R_2(A, B)$	a	b	c	d	$x_{2,5}$
$R_3(B, E)$	$x_{3,1} = \mathbf{a}$	b	c	d	e
$R_4(C, D, E)$	$x_{4,1} = \mathbf{a}$	$x_{4,2}$	c	d	e
$R_5(A, E)$	a	$x_{5,2}$	c	d	e

7. 发现表中有一行是 $(\mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d}, \mathbf{e})$, 说明该分解是无损连接分解

	A	B	C	D	E
$R_1(A, D)$	a	$x_{1,2}$	c	d	$x_{1,5}$
$R_2(A, B)$	a	b	c	d	$x_{2,5}$
$R_3(B, E)$	a	b	c	d	e
$R_4(C, D, E)$	a	$x_{4,2}$	c	d	e
$R_5(A, E)$	a	$x_{5,2}$	c	d	e

■ 无损连接性的判定 (续)

为什么表中有一行是(a, b, c, d, e)就能说明该分解是无损连接分解?

① 显然, $R \subseteq \Pi_{A,D}(R) \bowtie \Pi_{A,B}(R) \bowtie \Pi_{B,E}(R) \bowtie \Pi_{C,D,E}(R) \bowtie \Pi_{A,E}(R) = R_1 \bowtie R_2 \bowtie R_3 \bowtie R_4 \bowtie R_5$

② 往证 $R_1 \bowtie R_2 \bowtie R_3 \bowtie R_4 \bowtie R_5 \subseteq R$

▶ 对任意(a, b, c, d, e) $\in R_1 \bowtie R_2 \bowtie R_3 \bowtie R_4 \bowtie R_5$, 它是由以下元组连接而成的

★ (a, d) $\in R_1$

★ (a, b) $\in R_2$

★ (b, e) $\in R_3$

★ (c, d, e) $\in R_4$

★ (a, e) $\in R_5$

▶ 这些元组又是通过R中以下元组投影得到的($x_{i,j}$ 代表未知数)

★ (a, $x_{1,2}$, $x_{1,3}$, d, $x_{1,5}$) $\in R \xrightarrow{\text{projection}} (a, d) \in R_1$

★ (a, b, $x_{2,3}$, $x_{2,4}$, $x_{2,5}$) $\in R \xrightarrow{\text{projection}} (a, b) \in R_2$

★ ($x_{3,1}$, b, $x_{3,3}$, $x_{3,4}$, e) $\in R \xrightarrow{\text{projection}} (b, e) \in R_3$

★ ($x_{4,1}$, $x_{4,2}$, c, d, e) $\in R \xrightarrow{\text{projection}} (c, d, e) \in R_4$

★ (a, $x_{5,2}$, $x_{5,3}$, $x_{5,4}$, e) $\in R \xrightarrow{\text{projection}} (a, e) \in R_5$

▶ 根据例子, 我们知道($x_{3,1}$, b, $x_{3,3}$, $x_{3,4}$, e) = (a, b, c, d, e) $\in R$

■ 函数依赖保持性的判定

- 目的: 判定一个关系模式分解是否满足函数依赖保持性
- 方法1: 根据函数依赖保持性的定义进行判断
- 方法2: 对于函数依赖集 F 中每个函数依赖 $X \rightarrow Y$, 使用下面的算法判断是否 $F_1 \cup F_2 \cup \dots \cup F_n \models X \rightarrow Y$

函数依赖保持性判定

输入: 关系模式 R 的函数依赖集 F 、函数依赖 $X \rightarrow Y \in F$ 、 R 的分解 $\rho = \{R_1(U_1), R_2(U_2), \dots, R_n(U_n)\}$

输出: $X \rightarrow Y$ 是否被 $F_1 \cup F_2 \cup \dots \cup F_n$ 逻辑蕴含

```
1:  $X^+ \leftarrow X$ 
2: repeat
3:   for  $i = 1$  to  $n$  do
4:      $X^+ \leftarrow X^+ \cup ((X^+ \cap U_i)_F^+ \cap U_i)$ 
5: until  $X^+$ 不再变化
6: if  $Y \subseteq X^+$  then
7:   return true
8: else
9:   return false
```

■ 函数依赖保持性的判定 (续)

函数依赖保持性判定

```
1:  $X^+ \leftarrow X$ 
2: repeat
3:   for  $i = 1$  to  $n$  do
4:      $X^+ \leftarrow X^+ \cup ((X^+ \cap U_i)_F^+ \cap U_i)$ 
5: until  $X^+$  不再变化
6: if  $Y \subseteq X^+$  then
7:   return true
8: else
9:   return false
```

Example (函数依赖保持性判定)

SDN上的函数依赖集为 $F = \{Sno \rightarrow Sdept, Sdept \rightarrow Sdean\}$ 。
将SDN分解为SD(Sno, Sdept), SN(Sno, Sdean)

- ① 显然, SD保持了 $Sno \rightarrow Sdept$
- ② 因为 $\{Sdean\} \not\subseteq Sdept^+$, 所以 $Sdept \rightarrow Sdean$ 未被保持

■ 关系模式的分解方法

如何对一个设计得不够规范的关系模式进行分解？

- 人工分解：由数据库设计者根据关系模式上的函数依赖对关系模式进行人工分解
 - 优点：对关系模式的语义具有深入理解，不会“过度分解”
 - 缺点：只能对属性较少的关系模式进行分解
- 关系模式分解算法：使用算法对关系模式进行自动分解
 - 优点：能够对具有很多属性的关系模式进行自动分解
 - 缺点：缺少对关系模式语义的理解，可能会“过度分解”

■ 3NF 关系模式分解算法

- 功能: 将关系模式 $R(U, F)$ 分解为一组3NF关系模式, 且该分解既保持函数依赖, 又满足无损连接性

3NF关系模式分解算法

- 1: $G \leftarrow F$ 的最小覆盖
- 2: **if** 存在 $X \rightarrow Y \in G$ 使得 $X \cup Y = U$ **then**
- 3: **return** $\{R\}$
- 4: $\rho \leftarrow \{R_K(K)\}$ (K 是 R 的候选键)
- 5: $U_0 \leftarrow$ 不出现在 G 中任何函数依赖中的属性的集合
- 6: **if** $U_0 \neq \emptyset$ **then**
- 7: $\rho \leftarrow \rho \cup \{R_0(U_0)\}$
- 8: $i \leftarrow 1$
- 9: **for all** $X \rightarrow Y \in G$ **do**
- 10: $\rho \leftarrow \rho \cup \{R_i(X \cup Y)\}$
- 11: $i \leftarrow i + 1$
- 12: 从 ρ 中去除被其他关系模式包含的关系模式
- 13: **return** ρ

■ 3NF 关系模式分解算法 (续)

Example (3NF关系模式分解算法)

- 关系模式SDC(Sno, Sname, Sdept, Sdean, Cno, Grade)
- 函数依赖集 $F = \{Sno \rightarrow Sname, Sno \rightarrow Sdept, Sdept \rightarrow Sdean, (Sno, Cno) \rightarrow Grade\}$

F 的最小覆盖	分解后的关系模式	是否冗余
	$R_K(Sno, Cno)$	是
$Sno \rightarrow (Sname, Sdept)$	$R_1(Sno, Sname, Sdept)$	否
$Sdept \rightarrow Sdean$	$R_2(Sdept, Sdean)$	否
$(Sno, Cno) \rightarrow Grade$	$R_3(Sno, Cno, Grade)$	否

分解后的关系模式	范式	F 的投影
$R_1(Sno, Sname, Sdept)$	BCNF	$F_1 = \{Sno \rightarrow (Sname, Sdept)\}$
$R_2(Sdept, Sdean)$	BCNF	$F_2 = \{Sdept \rightarrow Sdean\}$
$R_3(Sno, Cno, Grade)$	BCNF	$F_3 = \{(Sno, Cno) \rightarrow Grade\}$

$F_1 \cup F_2 \cup F_3$ 覆盖 F , 所以保持函数依赖

■ BCNF 关系模式分解算法

- 功能: 将关系模式 $R(U, F)$ 分解为一组无损连接BCNF关系模式

BCNF关系模式分解算法

输入: 关系模式 $R(U, F)$

输出: R 的无损连接BCNF关系模式分解 ρ

- 1: $\rho \leftarrow \{R\}$
- 2: 计算 F^+ {指数时间复杂度}
- 3: **repeat**
- 4: **if** ρ 中存在关系模式 $R_i(U_i)$ 不属于BCNF **then**
- 5: 从 F_i^+ 中找出非平凡函数依赖 $X \rightarrow Y$, 满足 $X \cap Y = \emptyset$ 且 X 不是 R_i 的候选键
- 6: $\rho \leftarrow (\rho - R_i(U_i)) \cup \{R_{i1}(XY), R_{i2}(U_i - Y)\}$
- 7: **until** ρ 不再变化
- 8: **return** ρ



E-R 转模式



函数依赖



关系范式



模式分解